

Enterprise Data Compiler

Foundational Architecture Specification

Compiler-centered architecture for deterministic enterprise data contracts

Status	Pre-implementation architecture baseline
Version	0.3
Date	August 2026
Audience	Senior distributed systems, database, compiler, security, and platform engineers

This specification defines architecture, invariants, and decision protocols. It contains no implementation API or deployment configuration.

Contents

Part I — Foundations	5
1 Introduction	6
2 Industry History and Transferable Lessons	7
3 Why Current Architectures Fail	7
4 Design Principles and Non-goals	9
5 User Model	10
Part II — Semantic Model	12
6 Natural-Language Interaction	13
7 Intent Language	14
8 Integration Contracts	16
9 Enterprise Knowledge Graph	17
Part III — Compiler	19
10 Compiler Architecture	20
11 Abstract Syntax Tree	21
12 Intent Intermediate Representation	22
13 Semantic Validation and Static Analysis	23
14 Constraint Solving	24
15 Optimization	25
16 Cost Model	28
17 Feasibility Analysis	29
18 Execution Graph and Deployment Compilation	30
Part IV — Execution System	32
19 Runtime	33
20 Relay	34
21 Capability Providers	36
22 State Migration	38
23 Scalability	40
24 Failure Handling	41
Part V — Enterprise Control	43
25 Security Architecture	44
26 Governance	45
27 Operations	47
28 Explainability	49
29 AI Architecture	50
Part VI — Adoption and Evolution	52
30 Migration and Coexistence	53
31 Risks	54
32 Open Research Questions	55
33 Minimum Viable Platform	56
34 Future Evolution	58
Appendix A — Invariant Registry	60
Appendix B — State and Migration State Machines	61
Appendix C — Failure and Diagnostic Taxonomy	62
Appendix D — Glossary	63
Appendix E — Worked T0 Activation Summary	64
Appendix F — Technical References	65

Architecture Summary

This document specifies a pre-implementation architecture for a multi-tenant enterprise data compiler. It defines semantic artifacts, compilation stages, planning rules, execution responsibilities, security boundaries, state-migration protocols, and operational invariants. It deliberately does not specify implementation APIs, wire formats, service frameworks, or deployment manifests.

Requirements and design status

This is a pre-implementation specification. Stable requirements are identified by INV-*nn* and collected in Appendix A; inline citations identify their enforcement points. Other statements define the proposed v1 mechanism, a design target, or an open research question. Descriptive present tense does not imply that the system exists or that an unimplemented proof procedure has been validated.

Central claim

The permanent object is an Integration Contract. The compiler derives a versioned semantic Intent Intermediate Representation (IIR), admits a physical plan only through the v1 legality rules and feasibility evidence, and emits a disposable Execution Graph. The runtime executes the graph; it does not reinterpret the contract. If no considered plan satisfies the contract, compilation returns No Valid Plan with an unsatisfied-constraint proof. It never substitutes weaker semantics.

Table 1. Document artifact model

Artifact	Owner	Lifetime	Authority
Integration Contract	Enterprise user and policy authorities	Long-lived, versioned	Declares required outcomes and constraints
Semantic IIR	Compiler	Versioned with compiler and metadata snapshot	Canonical meaning used for analysis
Physical Plan	Planner	Replaceable after legality and stability checks	Selects capabilities, placement, and state strategy
Execution Graph	Deployment compiler	Disposable	Signed instructions for deterministic execution
Execution State	Named state owner	Durable until contract retirement	Offsets, checkpoints, deduplication, and migration epochs

Authority and lifetime of compiler artifacts

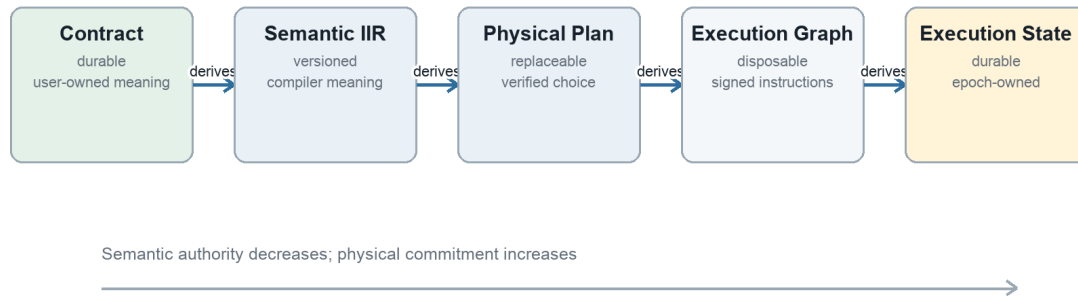


Figure 1. Artifact lifetimes and authority boundaries.

Part I — Foundations

1 Introduction

1.1 Problem definition

Enterprise data movement is usually represented as a directed pipeline whose nodes mix business meaning, connector mechanics, execution topology, scheduling, credentials, and recovery behavior. The representation is editable, but it is not a stable semantic contract. A change in connector, engine, or execution mode can therefore alter observable results without appearing to change business intent.

This architecture separates meaning from mechanism. A user declares required outcomes: which business records are in scope, the consistency and freshness boundary, retention, security treatment, permissible locations, consumer rights, and response to violations. Compilation admits only a physical strategy covered by a reviewed v1 legality rule and valid feasibility evidence. Runtime machinery is replaceable because its meaning has already been fixed.

1.2 System boundary

The platform owns compilation, plan verification, deployment compilation, contract-level state, and evidence that execution conforms to a contract. It integrates with existing catalogs, transformation frameworks, schedulers, warehouses, object stores, event systems, identity providers, and policy systems. It does not replace them. Integration, replication, synchronization, governed context serving, event routing, and operational data access are applications of the compiler rather than separate engines.

Table 2. *Primary objectives and acceptance signals*

Priority	Objective	Architectural acceptance signal
1	Simplicity	A tenant can deploy outbound-only relays without operating clusters or tuning distributed middleware.
2	Scale	The first-ten-customer design partitions compilation, coordination, evidence, and data execution independently without fixing a small upper bound.
3	Enterprise correctness	Every material decision has provenance, authorization, deterministic replay inputs, and an auditable outcome.
4	Natural-language interaction	Conversation produces a typed contract only after ambiguities and policy conflicts are resolved.

1.3 Non-goals

- No new enterprise catalog or glossary system.
- No warehouse, lakehouse, general transformation language, scheduler, or general-purpose workflow orchestrator.

- No promise of universal exactly-once behavior across endpoints that lack transactional or idempotent primitives.
- No hidden semantic degradation to meet cost, latency, or availability targets.
- No runtime dependence on probabilistic models for record-level correctness.

2 Industry History and Transferable Lessons

2.1 Comparable mechanisms and retained lessons

Table 3. Prior mechanism, observed boundary, and architectural response

System or mechanism	Specific documented behavior	Response in this design
MapReduce	A constrained map/reduce model separates user functions from distributed scheduling and recovery.	Keep runtime operators narrow; compile richer intent before execution.
Spanner	Externally consistent transactions depend on explicit uncertainty bounds and commit protocol.	Represent evidence validity and Unknown explicitly; do not infer guarantees from topology.
LLVM	A stable typed IR separates source languages from target-specific code generation.	Use typed IIR as the semantic boundary; execution graphs remain disposable.
SnapLogic Groundplex	Customer-managed execution nodes run configured pipeline processing in private networks.	Restrict the Relay to signed fragments, named epoch state, and no planning authority.
Fivetran Hybrid Deployment	A cloud control plane coordinates an agent that performs pipeline processing in the customer environment over outbound connectivity.	Retain outbound connectivity but reduce the local trust and operational surface.
Airbyte CDK	A connector development framework permits broad implementation variation behind a common protocol.	Treat provider declarations as claims; admit substitutions only with tiered conformance evidence.

The shared lesson is not that one historical system supplies the architecture. It is that execution flexibility is safe only behind an explicit semantic boundary: legality precedes cost (INV-02, INV-02a, INV-03).

3 Why Current Architectures Fail

3.1 Semantic and physical coupling

A pipeline node named replication, lookup, join, or query usually carries implicit assumptions about completeness, ordering, retry, null handling, deletion, and time. Replacing a mirror with a live query can change snapshot isolation, source load, failure independence, residency, and the meaning of freshness. Treating that replacement as an optimization is unsound.

3.2 Infrastructure as a customer concern

Cluster managers, brokers, worker farms, and self-hosted integration runtimes export capacity planning, patching, partition sizing, failure domains, and upgrade sequencing to customers. The platform then optimizes the vendor's software boundary rather than the user's data outcome. Outbound-only relays are sometimes unavoidable for private networks, but they must remain small, replaceable, and remotely attestable.

3.3 Metadata duplication

A platform that copies enterprise metadata into a proprietary catalog and makes that copy authoritative creates a second governance plane. Divergent ownership, tags, schemas, and lineage are inevitable. The compiler instead consumes versioned observations from existing authorities and records provenance, confidence, and staleness for each fact.

3.4 Optimizers without proof or stability

Rule-based switches often optimize a local metric while changing semantics or oscillating as measurements cross a threshold. A legal optimizer requires a semantic equivalence class, a feasibility proof, a cost model with uncertainty, shadow measurements where risk warrants them, hysteresis, minimum dwell time, and a migration protocol.

Table 4. *Failure pattern taxonomy*

Class	Typical symptom	Architectural correction
Semantic drift	Same authored pipeline, different results after engine change	Contract semantics and plan legality proof
Soft infeasibility	Freshness target silently missed	Compile-time and continuous feasibility analysis
Operational externalization	Customer maintains brokers and workers	Managed data plane plus minimal relay
Control-plane ambiguity	No record of why a plan changed	Decision record and signed plan lineage
State orphaning	Duplicate or missing records during migration	Epoch-based state ownership transfer
Probabilistic correctness	Generated transformation accepted without proof	Deterministic validation and test obligations

4 Design Principles and Non-goals

4.1 Ordered principles

- Simplicity is an architectural constraint: no customer-operated cluster is required for the normal case.
- Meaning precedes mechanism: semantic validation and feasibility precede optimization.
- No silent degradation: an unsatisfied hard constraint yields No Valid Plan or a running-contract violation.
- Contracts are durable; plans and graphs are replaceable.
- The compiler is authoritative for physical choice; the runtime is authoritative only for faithful execution.
- Catalogs remain external authorities; the Knowledge Graph is a compiler projection with provenance.
- AI may propose; deterministic systems validate, authorize, and execute.
- State has one named owner per contract partition and epoch.
- Automatic change is gated by legality, confidence, stability, and policy.

4.2 Architectural invariants

Invariants are assigned stable identifiers in Appendix A and cited at their enforcement points. Compiler passes, runtime components, operational procedures, and audits cannot substitute weaker local definitions.

Decision protocol — Automatic decision protocol

Required inputs: Contract version; semantic IIR; metadata snapshot; provider capability attestations; policies; statistics with age and confidence; current plan and dwell time; authorization mode.

Decision algorithm: Enumerate candidates; reject illegal candidates; reject infeasible candidates; calculate a multi-objective cost interval; compare against the incumbent including migration and risk; require confidence and stability thresholds; produce a signed decision record.

Invariants: Hard constraints are never relaxed. Semantic equivalence is established before cost comparison. The incumbent remains active until a replacement is verified and committed. Every input is version-addressed.

Failure modes: Missing or stale facts produce Unknown, not optimistic assumptions. Empty candidate sets produce No Valid Plan. Conflicting authorities produce an explicit ambiguity. Verification failure aborts the change.

Tradeoffs: Conservative decisions preserve correctness but can retain a more expensive plan. Stronger evidence and shadow execution cost resources but reduce unsafe adaptation.

4.3 Explicit exclusions

The platform is not a database, a universal transaction coordinator, a business-process engine, or a replacement for dbt and similar transformation systems. It may invoke those systems as capabilities when their semantics satisfy a contract. Human workflows, approvals, and incident processes are control-plane concerns, not a pretext for embedding a general scheduler.

5 User Model

5.1 Roles

A contract has a requester, data owner, policy authority, operator, and consumers. One person may hold several roles, but authorization is evaluated by role and scope. The requester states the outcome. The data owner attests source meaning and allowed uses. Policy authorities define non-negotiable controls. Operators respond to violations. Consumers receive data products, query access, events, resources, or tools under explicit entitlements.

5.2 Contract lifecycle

Conversation begins in Draft. A draft becomes Analyzable after references and material ambiguities are resolved. Compilation can produce No Valid Plan, Needs Approval, or a Verified Plan. Activation creates an epoch and assigns state ownership. Changes create new contract versions; they never mutate the active meaning in place. Retirement drains work, seals audit evidence, and applies retention policy to compiler-owned state.

Table 5. Human decision points

Decision	Default authority	Why it is not automatic
Resolve business ambiguity	Requester and data owner	Metadata cannot determine intended meaning
Override a policy conflict	Policy authority	Compiler cannot weaken an external control
Approve non-equivalent migration	Contract owner	Observable semantics may change
Authorize autonomous optimization	Tenant administrator	Changes spend, placement, and operational risk
Accept degraded service	Contract owner plus policy authority	A temporary relaxation changes the contract and must be versioned

5.3 Derived contract risk tier

Every contract carries a compiler-derived risk tier T0 through T3. The tier controls authoring ceremony, required approvals, verification depth, grace periods, and automation eligibility; it does not relax hard semantics or policy (INV-03). Users cannot lower it by assertion. Enterprise policy may raise it.

Table 6. V1 preferred-floor risk decision

First matching condition	Minimum tier	Reason
Regulated or secret data	T2	Governance failure has material legal or evidentiary consequence.
Any non-repeatable external effect	T2	Replay or ambiguous commit can cause an irreversible business effect.
Cross-border-restricted movement	T2	Placement and transfer legality require stronger proof and approval.
Production destination with more than one consumer	T1	A bad result has a broader operational blast radius.
None of the above	T0	Standard activation and verification ceremony is sufficient.

Decision protocol — Risk-tier derivation

Required inputs: Highest sensitivity classification; residency restriction; effect repeatability; destination environment; known consumer count; enterprise minimum-tier rules; versions and authorities for every input.

Decision algorithm: Evaluate all conditions in the preferred-floor table and take the maximum resulting floor. Apply any higher enterprise-policy floor. A prohibited path remains a policy conflict rather than a tier. Record the selected tier, matching predicates, decision-table version, input versions, and authorities.

Invariants: Missing, stale, or conflicting input never lowers the tier: use the conservative higher floor or block activation when no safe bound exists. Enterprise policy may raise but not lower the compiler floor. A tier change is a contract-impact event that re-evaluates approvals, verification, grace periods, and automation eligibility.

Failure modes: Hidden consumers, incorrect effect classification, or stale data classification can understate risk until revalidation. The dependency graph must trigger impact analysis when these facts change.

Tradeoffs: A floor table is coarse and deliberately conservative. It is easier to audit than additive scoring and avoids a false impression of quantitative precision, at the cost of grouping unlike contracts at the same tier.

Part II — Semantic Model

6 Natural-Language Interaction

6.1 Conversation is a source language, not an execution language

Natural language is parsed into candidate intent structures. The conversation service may infer terminology, map phrases to enterprise concepts, propose transformations, and ask focused questions. It cannot activate a contract, choose weaker semantics, or create an executable graph directly. The compiler accepts only a typed, provenance-bearing contract.

6.2 Ambiguity protocol

Ambiguity is material when alternatives differ in selected records, observable values, security, location, externally visible effects, or a non-defaultable policy. Material ambiguity blocks activation. Risk tier, rather than materiality alone, determines whether temporal, delivery, and governance choices require an answer before activation. For T0, the compiler may apply tenant-approved defaults when each value, authority, and rationale is recorded in contract provenance and surfaced in the activation summary and later review queue.

Decision protocol — Intent disambiguation

Required inputs: Conversation transcript; user identity and role; enterprise vocabulary; candidate dataset and field matches; policy facts; prior accepted contract patterns; confidence calibration for each candidate.

Decision algorithm: Generate a bounded set of interpretations; normalize each to a partial AST; compare their semantic deltas; eliminate interpretations that violate known policies; ask one question that maximally separates remaining material alternatives; repeat until one interpretation remains or the user cancels.

Invariants: Questions cannot reveal metadata the user is not authorized to see. Confidence never converts a material ambiguity into a default. Accepted answers are appended to contract provenance.

Failure modes: No authorized match, contradictory answers, or unresolved terminology leaves the draft Unanalyzable. Model failure falls back to explicit structured clarification; it never invents identifiers.

Tradeoffs: Additional questions slow authoring but prevent expensive semantic repair. Reusing prior patterns improves speed but risks institutionalizing outdated assumptions, so reuse remains advisory.

6.3 Example interpretation

The request to connect closed Salesforce opportunities to Snowflake with less than one minute freshness becomes predicates over opportunity state, a maximum observation delay, a retention obligation, a PII classification for email, an environment-dependent masking rule, a staleness alert threshold, and an entitlement for finance agents. Terms such as closed, production, finance, and retention authority must resolve through enterprise metadata or user confirmation.

Table 7. Conversation output classes

Output	Meaning	Can compile?
Clarification request	Material alternatives remain	No
Policy conflict	Requested outcome conflicts with a non-overridable control	No
Analyzable draft	Typed references and constraints are complete	Yes
Change proposal	Delta against an active contract is explicit	After impact analysis
Explanation	Human-readable account of an existing decision	Not applicable

6.4 Lowest-tier authoring and latency target

The T0 target is zero mandatory clarification questions when source, destination, table, and credentials resolve uniquely, and at most two questions otherwise. Tenant-approved defaults cover source-observation-to-destination-visibility freshness of 24 hours, at-least-once delivery with idempotent destination merge where available, source-aligned retention, a determined deletion behavior, non-production activation, and no consumer exposure beyond the requester. At activation the compiler records whether source deletion propagates, the provider evidence supporting that behavior, or the fact that it does not propagate; the activation summary exposes the value before data moves. If policy or stated intent requires deletion propagation and the target cannot provide it, the compiler asks one of the two permitted questions to select a capable target or revise the requirement under proper authority; otherwise it returns No Valid Plan. The target from first sentence to a running snapshot contract is under five minutes, excluding user-controlled authentication. Appendix E shows the complete summary for a worked T0 contract.

The interactive compilation response budget is p95 three seconds and p99 eight seconds for parse, authorized resolution, risk-tier derivation, semantic validation, bounded candidate generation, and cached legality/feasibility checks. Network probes, exhaustive solving, provider dry-runs, shadow execution, and deployment may continue asynchronously; the interface returns Analyzable, Compiling, Needs Approval, No Valid Plan, or Ready to Activate with phase-specific progress. Activation waits for all hard obligations even when the conversational response does not.

7 Intent Language

7.1 Denotational boundary

The intent language denotes an allowed set of observable histories, not a sequence of steps. An observable history includes records exposed to each consumer, declared ordering, visibility times, transformations, deletion effects, policy decisions, and violation signals. History containment is the

correctness specification: a legal physical plan produces only histories allowed by the contract (INV-02). The v1 admission mechanism is the sound but incomplete curated rule table in §15.2 (INV-02a). It rejects candidates whose containment cannot be established by a listed rule and discharged proof obligations; it may reject a legal plan, but must not admit an illegal one.

7.2 Constraint classes

Table 8. *Intent constraint classes*

Class	Examples	Default hardness
Selection	Entities, predicates, temporal range, deletes	Hard
Semantic transformation	Field mapping, normalization, derivation	Hard
Consistency and delivery	Snapshot boundary, ordering, duplication, replay	Hard
Service level	Freshness, recovery time, availability	Hard unless explicitly tiered
Governance	Retention, purpose, residency, masking, lineage	Hard
Exposure	Dataset, query, event, MCP resource or tool	Hard
Economic preference	Spend ceiling, preferred provider, carbon objective	Hard ceiling plus soft ordering
Operational response	Alert, pause, quarantine, escalation	Hard

7.3 Time and uncertainty

Freshness is defined against an explicit event: source commit, source observation, capture, destination visibility, or consumer readability. The contract records the clock domain and tolerated uncertainty. A claim such as less than one minute is invalid until the start and end events are fixed. Statistics and catalog observations carry observation time, validity interval, and uncertainty.

7.4 Composition

Contracts may import named semantic definitions, policies, and data products by immutable version reference. Imports are expanded for analysis but preserved for provenance. Cycles in semantic definitions are illegal unless the construct has a defined least fixed point and the compiler can prove termination; the MVP excludes recursive data derivations.

8 Integration Contracts

8.1 Contract contents

A contract contains identity and ownership, compiler-derived risk tier, source and consumer bindings, semantic selection, transformations, consistency and delivery obligations, temporal semantics, governance controls, service objectives, economic constraints, failure policy, evidence requirements, change policy, and authorization mode. It references enterprise concepts rather than copying their definitions.

Table 9. *Contract sections and validation authority*

Section	Primary authority	Validation
Business semantics	Data owner and enterprise vocabulary	Concept and schema resolution
Security and purpose	Policy systems and policy authority	Static information-flow and entitlement checks
Service objectives	Contract owner	Capability and capacity feasibility
Delivery semantics	Contract owner plus endpoint capabilities	End-to-end protocol proof
Economics	Tenant administrator	Forecast and hard budget admission
Change policy	Contract owner	Compatibility and approval-mode checks

8.2 Versioning and compatibility

Contract versions are immutable. A new version is backward compatible for a consumer only if it preserves that consumer's allowed histories, schema compatibility rule, entitlement, and service objectives. Adding a nullable field may be structurally compatible yet violate purpose limitation; removing a field may be acceptable for one consumer and breaking for another. Compatibility is therefore consumer-scoped and policy-aware.

8.3 Runtime conformance

Activation creates a conformance envelope containing the contract version, IIR version, plan digest, metadata snapshot identifiers, provider attestations, state epoch, and verification evidence. Runtime events are evaluated against that envelope. A plan cannot claim conformance after an input it depends upon expires; it enters Revalidation Required and follows the contract's safety policy.

Decision protocol — Contract change classification

Required inputs: Old and new contract versions; affected consumers; schema and semantic deltas; policy deltas; active state layout; provider capabilities; tenant change policy.

Decision algorithm: Compute a typed semantic diff; determine each consumer's history inclusion relation; run governance impact analysis; classify as equivalent, backward compatible, conditionally compatible, or breaking; attach required migration and approvals.

Invariants: A textual similarity score is irrelevant. Unknown impact is not compatible. Active meaning changes only at an epoch boundary.

Failure modes: Incomplete lineage or unresolved consumers yields Unknown and blocks autonomous activation. A falsely broad equivalence rule can corrupt consumers; equivalence rules are versioned and regression-tested.

Tradeoffs: Conservative classification increases approvals. Precise consumer-scoped analysis requires richer lineage and contracts.

9 Enterprise Knowledge Graph

9.1 Purpose and boundary

The Enterprise Knowledge Graph is a compiler data structure, not a new catalog. It projects facts from existing catalogs, policy systems, identity systems, provider discovery, runtime measurements, and approved contracts into a typed graph optimized for resolution and analysis. Authoritative metadata remains in the source system.

Unlike a proprietary catalog embedded in an integration product, this projection does not ask an enterprise to re-author ownership or classification already maintained in Unity Catalog, AWS Glue, OpenMetadata, DataHub, Atlan, or Collibra. It stores compiler-specific bindings and provenance, not a competing source of truth (INV-11).

9.2 Fact model

Each graph fact carries subject, predicate, object, authority, retrieval identity, observed time, validity interval, confidence class, tenant boundary, and visibility policy. Derived facts name the rule and input fact versions. Conflicting facts are retained; authority precedence is applied by predicate and scope rather than by ingestion order.

Table 10. Graph node families

Family	Examples	Compiler use
Business	Concepts, domains, owners, purposes	Name resolution and policy scope
Data	Datasets, schemas, fields, statistics, snapshots	Typing, mapping, feasibility, cost
Policy	PII, residency, retention, entitlements	Static analysis and placement constraints
Capability	Providers, operations, semantics, health	Candidate generation and feasibility
Operational	Plans, deployments, incidents, measurements	Adaptation and explanation

Family	Examples	Compiler use
Consumer	Applications, agents, tools, resources	Exposure and purpose limitation

Decision protocol — Metadata conflict resolution

Required inputs: Conflicting facts; predicate-specific authority ordering; scope; freshness limits; source trust; contract-pinned facts; user authorization.

Decision algorithm: Group facts by semantic key; discard facts outside validity; apply scope-specific authority precedence; if a unique highest-authority fact remains, select it; otherwise preserve an ambiguity and request adjudication. Derived facts inherit the weakest confidence and shortest validity of their inputs.

Invariants: No last-writer-wins across independent authorities. Contract-pinned facts cannot change silently. Selection is reproducible from versioned inputs.

Failure modes: Unavailable authority produces stale or Unknown facts. Equal-authority conflict blocks dependent compilation. Excessive invalidation can cause compilation pressure and is controlled through snapshotting and bounded refresh.

Tradeoffs: Strict provenance raises storage and invalidation costs. Cached projections improve compilation latency but require explicit staleness bounds.

9.3 Consistency model

A compilation reads a repeatable metadata snapshot assembled from source-specific versions or observations. Global simultaneity across catalogs is neither assumed nor required. The compiler records the observation interval and rejects plans when cross-source skew exceeds a constraint relevant to correctness. Runtime revalidation can use newer snapshots but cannot retroactively alter the meaning of an activated epoch.

Part III — Compiler

10 Compiler Architecture

10.1 Phase separation

Compilation is a sequence of pure or explicitly effectful phases. Parsing, canonicalization, typing, semantic analysis, constraint construction, candidate generation, legality checking, feasibility, optimization, verification, and deployment compilation have separate inputs and outputs. A later phase cannot repair an invalid earlier artifact by guessing. Legality and feasibility exclude candidates before cost ranking (INV-02, INV-03, INV-04).

Compilation pipeline and rejection boundaries

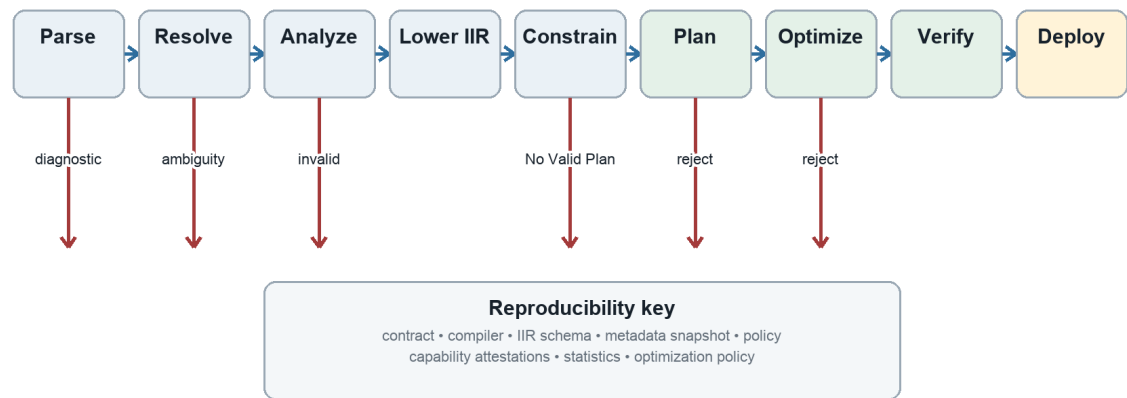


Figure 2. Compiler phases and rejection boundaries.

Table 11. Compiler passes

Pass	Consumes	Produces or rejects
Parse	Conversation-derived intent or imported source	AST with source provenance
Resolve	AST plus Knowledge Graph snapshot	Bound identifiers or ambiguity
Semantic analysis	Bound AST	Typed canonical AST or diagnostics
Lowering	Canonical AST	Semantic IIR
Static analysis	IIR plus policy facts	Obligations and violations
Constraint construction	IIR plus capabilities	Solver model
Planning	Solver model and statistics	Legal feasible candidates
Optimization	Candidates and cost intervals	Selected plan plus explanation

Pass	Consumes	Produces or rejects
Verification	Plan, provider attestations, test evidence	Verified plan or rejection
Deployment compilation	Verified plan	Signed execution graph and manifest

10.2 Reproducibility

Every compilation is identified by the contract version, compiler version, IIR schema version, Knowledge Graph snapshot, policy bundle, provider capability versions, statistics snapshot, legality-rule-table version, and optimization policy. Recompile with identical inputs yields an equivalent diagnostic set and deterministically tie-broken plan (INV-09). External probes are captured as inputs rather than performed invisibly inside pure passes.

10.3 Incrementality

The compiler memoizes pass results by content address and typed dependency set. Schema and policy changes are first tested for structural non-interference; only affected proof dependencies are invalidated. Statistics changes ordinarily invalidate cost ranking, while capability semantic changes invalidate legality and feasibility. Bounded mass revalidation is specified in §27.5.

11 Abstract Syntax Tree

11.1 AST responsibilities

The AST preserves user-visible structure and provenance while eliminating linguistic surface variation. It represents references, selection, temporal boundaries, transformations, service objectives, governance constraints, exposures, failure responses, and unresolved alternatives. It does not contain provider names, partitions, workers, queues, or retry topology unless the user explicitly declared one as a hard placement constraint.

11.2 Resolution and typing

Names initially bind to candidate graph nodes. Resolution considers authorization before ranking; an unauthorized asset must not appear as an alternative. Types include scalar and structural data types, event-time and system-time annotations, sensitivity labels, location labels, nullability, cardinality bounds, and effect types for reads, writes, actions, and external exposure.

Table 12. *AST diagnostic classes*

Diagnostic	Example	Disposition
Unbound reference	Finance is not mapped to a governed consumer group	Clarify or bind
Ambiguous semantic	Closed means won or any terminal stage	Clarify
Type error	Masking function incompatible with nested object	Correct intent
Policy contradiction	Five-year retention exceeds regional maximum	Reject or authorized exception
Incomplete temporal meaning	Freshness lacks start event	Clarify
Unauthorized reference	Requester cannot discover source schema	Reject without disclosure

11.3 Canonicalization

Canonicalization sorts commutative predicates, expands controlled vocabulary aliases, normalizes time units and locations, and records explicit defaults. It does not rewrite semantically meaningful order or collapse distinct null, missing, and deleted states. Canonical AST equality is a syntactic aid, not a proof of semantic equivalence.

12 Intent Intermediate Representation

12.1 Why a separate IIR

The IIR is the stable boundary between source languages and physical planners. Natural language, imported legacy definitions, declarative infrastructure descriptions, and MCP-originated requests may lower to the same IIR if they denote the same contract. The IIR is not a universal interchange promise in the MVP; compatibility is earned by formal semantics and conformance suites, not by accepting arbitrary syntax.

12.2 Layered representation

Table 13. *IIR layers*

Layer	Content	Permitted transformations
Semantic IIR	Typed relations, effects, time, policy, delivery obligations	Meaning-preserving normalization
Logical IIR	Abstract operators and state requirements	Algebraic rewrites with proofs
Distributed IIR	Partitioning, exchange, checkpoints, fault domains	Legal physical alternatives
Deployment IIR	Provider bindings, placement, resources, signed artifacts	Target-specific lowering only

12.3 Semantic identity and evolution

Each IIR node has a stable semantic identity derived from its normalized operation, bound concepts, and relevant constraints. Physical attributes do not affect semantic identity. IIR versions are immutable and declare reader compatibility. A compiler upgrade must either translate an older IIR with a verified migration or recompile from the contract and original provenance.

12.4 Effects and obligations

Operations carry effects such as reads external state, writes durable state, emits externally visible action, handles PII, crosses location, or requires nondeterministic external ordering. Obligations such as idempotency, deduplication, encryption, audit emission, deletion propagation, and human approval flow with values through the representation and must be discharged by a later node or remain as deployment requirements.

13 Semantic Validation and Static Analysis

13.1 Validation order

Validation proceeds from name and type correctness to temporal semantics, information flow, delivery realizability, retention compatibility, consumer entitlement, and change compatibility. Diagnostics are accumulated when safe so the user receives a coherent set, but security-sensitive errors are redacted according to authorization.

Table 14. *Static analyses*

Analysis	Property established	Representative failure
Type and shape	Transformations accept and produce declared structures	Unsafe narrowing or missing required field
Temporal	Event-time, observation-time, and visibility constraints are coherent	Freshness measured from incomparable clocks
Information flow	Sensitive values reach only authorized sinks in allowed form	Unmasked PII crosses environment boundary
Delivery	Required ordering and duplicate behavior can be composed	Sink lacks idempotency for retryable writes
Retention	Copies, logs, checkpoints, and evidence respect policy	Replay log retained longer than source purpose
Lineage completeness	Required provenance can be emitted across all operators	Opaque action has no result lineage
Determinism	Replay produces equivalent outputs for pinned inputs	External lookup lacks version or snapshot boundary

13.2 Proof obligations

Static analysis does not merely attach warnings. It emits obligations that must be discharged by provider attestations, runtime guards, verification tests, or explicit contract constraints. An obligation is closed only by evidence with matching scope and validity. Unclosed hard obligations make the plan infeasible.

13.3 Generated transformation validation

A model may propose mappings or expressions, but the accepted artifact is parsed, typed, normalized, analyzed for effects, tested against examples and boundary cases, and approved under tenant policy. Generated prose is not executable evidence. If the transformation uses an inherently probabilistic operation, its output must be classified as probabilistic data with an explicit acceptance policy; it cannot masquerade as deterministic integration.

14 Constraint Solving

14.1 Solver model

Planning is a constrained search over capability choice, execution mode, placement, partitioning, consistency mechanism, state store, exposure method, and failure policy. Hard constraints derive from contract semantics, policy, provider semantics, locality, and verified capacity. Soft objectives order otherwise legal plans.

Table 15. *Decision variables*

Variable family	Examples	Key constraints
Acquisition	Snapshot, CDC, polling, query	Completeness, freshness, source support
Processing	Pushdown, managed operator, existing framework	Semantics, determinism, location
Materialization	Mirror, cache, direct query, event	Consistency, retention, availability
Placement	Region, trust zone, relay, managed plane	Residency, egress, network reachability
State	Checkpoint store, dedupe index, replay log	Ownership, durability, retention
Exposure	Dataset, query, MCP resource, MCP tool	Authorization, latency, side effects

Decision protocol — Constraint solution selection

Required inputs: Semantic IIR; policy constraints; capability declarations; placement graph; capacity bounds; state compatibility; operational mode.

Decision algorithm: Construct a finite candidate domain; apply propagation and domain reduction; solve hard constraints; extract a minimal or near-minimal unsatisfiable core if none; enumerate a bounded Pareto frontier for optimization.

Invariants: Only declared and verified capabilities enter the domain. Solver timeouts do not imply infeasibility. A returned candidate includes a proof trace from each hard constraint to satisfying choices.

Failure modes: Unsatisfiable yields No Valid Plan. Timeout yields Planning Inconclusive with retained incumbent if one exists. Inconsistent capability data invalidates affected candidates.

Tradeoffs: Complete search can be expensive. Bounded domains and staged solving improve latency but may miss exotic plans; the compiler reports search bounds and supports offline exhaustive analysis for critical contracts.

14.2 No Valid Plan

No Valid Plan is a first-class result containing the conflicting constraints, relevant unavailable capabilities, staleness of evidence, and the smallest known changes that would restore feasibility. Suggested relaxations are proposals only. They never alter the contract.

15 Optimization

15.1 Optimization after legality

The optimizer sees only candidates admitted by the v1 legality checker and proven feasible. Transformations are organized by equivalence class: relational rewrites, predicate and projection placement, partitioning and exchange, materialization strategy, batching, and provider selection. History containment remains the correctness criterion (INV-02); v1 enforces it through the deliberately incomplete rule-table mechanism (INV-02a). Cost cannot rescue a rejected or Unknown legality result (INV-03, INV-04).

15.2 V1 plan-legality checker

V1 uses a curated, versioned rule table rather than an automated equivalence prover (INV-02a). Rule authors discharge semantic proof obligations in written proof notes and provider-pair conformance tests. The compiler matches a candidate substitution to an exact rule version, instantiates its preconditions from the contract and provider attestations, and rejects the candidate if the rule is absent, any obligation is Unknown, or required evidence is invalid. A rule can be added or widened only with a reviewed proof note, positive and negative conformance fixtures, mutation tests for each precondition, and regression against every previously admitted pair. Soundness is required; completeness is not.

Table 16. *Equivalence dimensions used by every legality rule*

ID	Dimension	Required comparison
D1	Record set and deletion	Selection, intermediate states, tombstones, and completion
D2	Value semantics	Null/missing, precision, collation, locale, and time zone
D3	Temporal visibility	Snapshot, isolation, ordering, event time, and visibility boundary

ID	Dimension	Required comparison
D4	Duplicate and replay	Retry envelope, idempotency, and deterministic replay
D5	Failure coupling	Availability dependence and failure independence
D6	Governance	Security, purpose, residency, retention, and consumer authorization
D7	Operational side effects	Source load, quota, and externally visible effects constrained by contract
D8	Evidence	Lineage, audit events, measurements, and proof provenance

Table 17. V1 execution-mode substitution rule table

Rule	Substitution / default	D1-D8 proof obligations
L01	Snapshot mirror -> incremental mirror / Conditional	D1 common snapshot and complete changes incl. deletes; D2 equal decoding; D3 cut boundary and order; D4 replay/dedupe; D5 retained-log failure envelope; D6 same copy location/retention; D7 CDC load within contract; D8 source positions and lineage.
L02	Incremental mirror -> snapshot mirror / Conditional	D1 same final set and delete state; D2 equal decoding; D3 bounded pause or dual-read boundary; D4 state handoff; D5 availability no weaker; D6 copy permitted; D7 snapshot load admitted; D8 cutover evidence.
L03	Mirror -> live query / Reject unless contract owner and policy authority accept	D1 predicate equivalence; D2 query value semantics; D3 declared query isolation/visibility; D4 read replay semantics; D5 source coupling accepted; D6 no forbidden materialization change; D7 load ceiling; D8 query audit/lineage. Acceptance is provenance-bound to both authorities and the contract version.
L04	Live query -> mirror / Conditional	D1 complete copy and deletes; D2 materialization equivalence; D3 freshness boundary; D4 replay/dedupe; D5 independence no weaker; D6 retention/residency permit copy; D7 source load admitted; D8 copy lineage.
L05	Polling -> CDC / Conditional	D1 equal selection and deletes; D2 equal decode; D3 starting position/order; D4 duplicate envelope; D5 retained-log behavior; D6 equal controls; D7 CDC quota; D8 positions and events.
L06	CDC -> polling / Reject unless lossy history allowed	D1 missed intermediate states explicitly allowed; D2 equal decode; D3 weaker order/freshness allowed; D4 polling replay proven; D5 polling coupling allowed; D6 equal controls; D7 query load admitted; D8 observation evidence.
L07	Managed processing -> source pushdown / Conditional	D1 predicate/operator equivalence; D2 null/collation/time-zone/precision equivalence; D3 source snapshot boundary; D4 deterministic replay; D5 source failure coupling accepted; D6 policy enforcement equivalent; D7 source load admitted; D8 explain/lineage evidence.
L08	Batch -> streaming / Conditional	D1 same window membership/deletes; D2 equal values; D3 event-time/late-data/order/visibility; D4 commit and replay; D5 broker dependencies accepted; D6 equal controls; D7 sustained load admitted; D8 window/offset evidence.
L09	Streaming -> batch / Reject unless batching allowed	D1 same final set; D2 equal values; D3 freshness/intermediate visibility permits batches; D4 batch replay; D5 schedule dependence accepted; D6 equal controls; D7 batch spikes admitted; D8 batch boundary evidence.

Rule	Substitution / default	D1-D8 proof obligations
L10	Dataset exposure -> MCP resource / Conditional	D1 result-set bounds; D2 schema/value equivalence; D3 freshness contract; D4 read replay behavior; D5 service coupling accepted; D6 authorization/purpose equivalent; D7 query limits; D8 provenance and access evidence.
L11	Resource read -> MCP tool action / Illegal	D1-D8 cannot establish equivalence because the effect class changes; a new contract and authorization are required.

Every admitted substitution carries a proof note identifying rule version, instantiated D1-D8 obligations, evidence digests, reviewer, conformance-suite version, and known applicability limits. The compiler emits No Valid Plan or retains the incumbent when the table has no matching rule; it never infers a new rule from model output or similarity (INV-02a, INV-04, INV-09, INV-10).

15.3 Adaptive modes

Table 18. *Optimization modes*

Mode	Behavior	Required authority
Recommendation	Ranks alternatives and explains expected effect; no change	Default
Approval	Prepares verified migration; waits for named approver	Tenant policy
Autonomous	May execute equivalent migration within risk and budget envelope	Explicit tenant grant

Decision protocol — Adaptive reoptimization

Required inputs: Current verified plan; legal candidate set; recent and historical measurements; cost intervals; confidence; contract risk class; minimum dwell time; migration readiness; authorization mode.

Decision algorithm: Filter candidates by semantic equivalence to the incumbent contract; require improvement beyond hysteresis across the configured observation window; include migration cost and rollback exposure; require confidence threshold; shadow high-risk candidates; propose or execute according to mode.

Invariants: No oscillation within minimum dwell time except safety events. Cost improvement cannot justify weaker semantics. Exactly one plan owns each state partition in an epoch.

Failure modes: Measurement drift can produce false improvements; shadow results can be unrepresentative; provider changes can invalidate a candidate. Any failed precondition leaves the incumbent untouched.

Tradeoffs: Hysteresis sacrifices short-term savings for stability. Shadow execution consumes duplicate resources. Autonomous mode reduces operator load but is restricted to narrow equivalence classes.

15.4 Shadow execution

Shadow execution receives a bounded copy of inputs or replays immutable input ranges, writes to isolated sinks, and compares semantic outputs, latency distributions, resource use, and failure behavior.

It never invokes externally visible actions. Comparison tolerances are contract-specific; unordered results require canonical multiset comparison, while time-dependent results require pinned clocks or recorded external observations.

16 Cost Model

16.1 Multi-objective cost

Cost is a vector, not a scalar invoice estimate. It includes provider charges, compute, storage, data movement, source load, relay load, control-plane overhead, expected failure recovery, migration cost, operational toil, and risk penalty. Hard budget ceilings remain constraints; other dimensions form a tenant-configured ordering or Pareto frontier.

Table 19. *Cost inputs and uncertainty*

Input	Source	Uncertainty treatment
Cardinality and size	Catalog statistics and runtime measurements	Age-weighted intervals; skew model
Change rate	CDC offsets or observation history	Seasonal quantiles and burst factor
Selectivity	Histograms, samples, shadow probes	Confidence interval and correlation caveat
Provider pricing	Versioned pricing facts	Effective dates and tier boundaries
Latency and throughput	Per-capability measurements	Tail distributions, not averages
Failure rate	Operational history by fault domain	Bayesian prior for sparse evidence
Migration	State volume and protocol	Worst-case transfer plus dual-run window

16.2 Confidence

Every estimate is an interval with provenance and confidence. The optimizer uses risk-adjusted dominance: a candidate does not dominate the incumbent when confidence intervals overlap within the tenant's risk tolerance. Missing statistics do not become zero cost; they receive conservative priors or trigger measurement.

16.3 Economic alignment

A vendor that earns more when customer compute rises has an incentive conflict. The architecture therefore separates measured resource consumption, customer-visible optimizer rationale, and commercial pricing. Preferred pricing is based on contract management, verified data volume, or reserved service envelopes rather than opaque compute markup. Tenant policy may pin a spend ceiling,

require recommendation mode, or audit plan alternatives. The optimizer must expose when a cheaper legal plan was rejected and why.

Decision protocol — Statistics acquisition

Required inputs: Missing estimate; contract risk class; source permissions; sampling cost; data sensitivity; allowed probe methods; current uncertainty.

Decision algorithm: Choose the least invasive measurement that can reduce decision uncertainty below threshold: reuse recent measurements, metadata statistics, bounded sample, read-only explain facility, or shadow execution. Stop when expected value of information falls below measurement cost.

Invariants: Measurement respects purpose, residency, and source-load limits. Samples are never retained longer than the contract allows. Absence of permission preserves Unknown.

Failure modes: Biased samples, correlated predicates, or transient load can mislead. Measurement failure does not invalidate the incumbent plan.

Tradeoffs: More measurement improves decisions but adds source load, delay, and cost. Conservative priors may select a costlier initial plan.

17 Feasibility Analysis

17.1 Feasibility dimensions

A legal plan may still be infeasible because capacity, network reachability, source quotas, retention storage, operational limits, or required evidence are unavailable. Feasibility is evaluated at compile time, activation admission, and continuously at runtime. It is distinct from optimization.

Table 20. Feasibility result

Result	Meaning	Action
Proven feasible	All hard obligations have valid evidence and capacity envelope	May optimize and verify
Conditionally feasible	Named assumption has bounded validity	Activate only if contract permits guard and fallback
Inconclusive	Evidence missing or analysis timed out	Measure or await evidence
Infeasible	At least one hard constraint cannot be met	No Valid Plan

Decision protocol — Freshness feasibility

Required inputs: Source commit semantics; acquisition latency distribution; batching; transfer bandwidth; processing latency; destination commit and visibility latency; clock uncertainty; failure budget; target percentile and threshold.

Decision algorithm: Compose tail-latency bounds along the critical path, accounting for correlated stages and retry budget; validate with probes or shadow execution when analytic confidence is insufficient; reserve capacity at admission.

Invariants: The freshness start and end events match the contract. Percentile composition never uses sums of averages. Conditional assumptions have monitors and expiry.

Failure modes: Unbounded provider latency, missing source commit time, or insufficient capacity makes the objective infeasible or inconclusive. Runtime violations trigger the declared response.

Tradeoffs: Conservative tail bounds may reject workable plans. Empirical validation costs time and load but is required for tight objectives.

17.2 Continuous admission

Activation reserves a logical capacity envelope, not dedicated machines. Runtime envelopes are not oversubscribed for T3 contracts or non-preemptible hard objectives. T0-T2 burst capacity may be oversubscribed only from measured, seasonally adjusted simultaneous-use quantiles, within a tenant risk ceiling, and with at least 20% uncommitted regional headroom. On simultaneous claim, reserved non-preemptible work is admitted first; lower-tier elastic work is backpressured or preempted only under terms already present in its contract. A missed reservation is a feasibility violation, not an implicit relaxation (INV-03).

18 Execution Graph and Deployment Compilation

18.1 Execution Graph

The Execution Graph is a signed, immutable artifact for one contract version, plan version, and state epoch (INV-01). It contains deterministic operators, capability bindings, partitioning, state references, retry and commit protocols, resource envelopes, observability obligations, and rollout guards. It contains no natural-language interpretation step.

18.2 Deployment compilation

Deployment compilation maps the verified physical plan to managed runtime cells and optional customer relays. It chooses failure domains, resource classes, network routes, secret references, update rings, and health checks. These choices cannot introduce new data access or cross a trust boundary not present in the verified plan.

Table 21. *Deployment artifact properties*

Property	Requirement
Integrity	Content-addressed and signed by the tenant-scoped compiler authority
Authorization	Carries least-privilege capability grants, never raw credentials

Property	Requirement
Compatibility	Declares minimum runtime and provider versions
Rollback	Names previous graph, state compatibility, and rollback deadline
Observability	Declares required metrics, traces, evidence, and redaction
Expiry	Invalid after contract, policy, provider attestation, or secret lease expiry

18.3 Verification

Verification includes structural checks, obligation closure, provider conformance, deterministic test vectors, fault injection for delivery protocols, security-policy checks, and deployment dry-run. For high-risk changes, a shadow or canary epoch compares output and operational envelopes before ownership moves.

Part IV — Execution System

19 Runtime

19.1 Runtime responsibilities

The runtime loads a signed graph, obtains short-lived capability grants, executes deterministic operators, maintains leased state ownership, emits evidence, and reports conformance. It does not choose business mappings, relax objectives, or select a new physical strategy. When an obligation cannot be met, it stops or degrades only as the contract explicitly permits.

Deployment topology

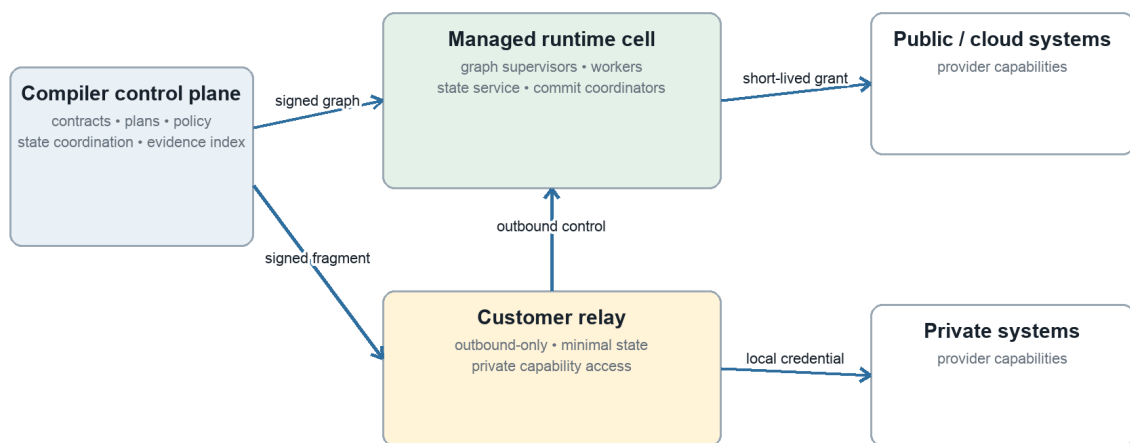


Figure 3. Managed control plane, regional runtime cells, customer relay, and external systems.

Table 22. Runtime component model

Component	State	Failure behavior
Graph supervisor	Ephemeral plus graph digest	Restarts from durable epoch state
Operator worker	Ephemeral	Retry or replacement under graph protocol
State service	Durable checkpoints and dedupe	Quorum or provider durability semantics
Commit coordinator	Durable intent for multi-step delivery	Completes, compensates, or surfaces indeterminate
Evidence emitter	Append-only buffered evidence	Backpressure or fail-closed by contract
Health agent	Ephemeral observations	Cannot declare semantic success alone

19.2 Boring by construction

Operators are narrow and deterministic: read, filter, project, transform, partition, exchange, checkpoint, commit, publish, and verify. Complex provider behavior remains behind a conformance-tested capability boundary. Runtime cells are homogeneous enough for rapid replacement and do not require tenant tuning.

19.3 Backpressure and overload

Backpressure propagates from destination commit capacity to acquisition within source-specific safety rules. The runtime bounds memory and in-flight work per contract partition. It spills only to approved encrypted state and accounts spilled data under retention. Priority and fairness are enforced by tenant and contract class; a noisy tenant cannot consume another tenant's reserved envelope.

20 Relay

20.1 Purpose

A relay exists only when managed runtime cannot reach a private endpoint. It is outbound-only, container-native, minimally stateful, auto-updated, and incapable of independent planning. It receives signed graph fragments scoped to local capabilities, exposes no public listener, cannot widen plan access (INV-12), and never receives raw secrets through compiler artifacts (INV-13).

Table 23. Relay trust invariants

Area	Invariant
Identity	Hardware- or OS-backed relay identity is tenant-bound and rotatable
Work authorization	Every work unit references a signed graph fragment and unexpired lease
Secrets	Short-lived endpoint credentials are resolved locally or delivered in sealed form
State	Relay caches are bounded; durable ownership is explicit and recoverable
Updates	Binaries and policy bundles are signed, staged, attestable, and rollbackable
Network	Only declared destinations and control endpoints are reachable

Decision protocol — Relay update rollout

Required inputs: Current and target signed versions; runtime compatibility; active graph requirements; relay health; tenant maintenance policy; rollback artifact; vulnerability severity.

Decision algorithm: Verify signature and provenance; check graph compatibility; stage inactive binary; run self-tests; drain or checkpoint affected work; switch atomically; monitor canary window; roll back on invariant breach. Emergency security updates may shorten dwell time under pre-authorized policy.

Invariants: No unsigned code executes. Active work retains exactly one state owner. Update cannot widen network or secret scope. Rollback remains possible until state compatibility deadline.

Failure modes: Power loss during switch restarts from the last committed epoch. Incompatible critical update can quarantine the relay and return contracts to No Valid Plan. Control-plane loss never authorizes stale new work.

Tradeoffs: Automatic updates reduce exposure but can disrupt private environments. Slow rings improve safety but extend vulnerability windows.

20.2 Offline behavior

A disconnected relay may finish already leased, bounded work when its graph and credentials remain valid (INV-20). It cannot accept new contract versions or extend leases. Evidence is buffered within a declared limit; exceeding that limit follows §27.4.

20.3 Comparison with self-hosted integration runtimes

Table 24. Relay boundary compared with named hybrid runtimes

Concern	Relay	SnapLogic Groundplex / Fivetran Hybrid Agent
Planning authority	None; executes signed fragments only	Runs configured pipeline processing in the customer network
Durable state	Contract-partition owner and epoch; bounded local cache	Runtime-specific local processing and storage model
Credentials	Local resolution or sealed short-lived capability grant	Connection credentials or agent token/configuration under product controls
Updates / rollback	Signed, staged, graph-compatible, state-aware rollback deadline	Installed node or agent lifecycle controlled by the product's deployment model
Network	Outbound control; per-plan egress allow-set; no public listener	Groundplex can require customer load balancing for inbound-triggered workloads; Fivetran agent uses outbound mTLS

A SnapLogic Groundplex is a self-managed data plane with execution nodes; a Fivetran Hybrid Deployment Agent performs local pipeline processing under a cloud control plane. The Relay is intentionally less capable: it cannot author or plan, accept work outside a signed graph fragment, or own durable state except as named by the contract epoch. This reduces customer infrastructure and compromise scope but does not support arbitrary local orchestration.

21 Capability Providers

21.1 Provider model

A Capability Provider describes operations an external system can perform and the semantics under which they are valid. It can expose read, write, change capture, events, metadata, query, actions, resources, tools, schemas, health, and policy-related capabilities. The compiler selects capabilities; users do not assemble source and destination connector nodes.

Unlike connector SDKs in Airbyte, SnapLogic, or similar platforms, the provider boundary is not primarily an implementation hook that yields a runnable connector. Its product is a set of typed semantic claims, limits, placement constraints, and evidence that the compiler may combine. Implementation code remains necessary, but it is not planner authority and is quarantined when observed behavior contradicts its declaration.

Table 25. *Capability declaration dimensions*

Dimension	Examples
Operation	Snapshot read, incremental read, transactional write, query, action
Semantics	Ordering scope, isolation, duplicates, deletion, replay, idempotency
Limits	Rate, concurrency, payload, retention, partition count
Placement	Regions, private reachability, execution locality
Security	Auth modes, secret handling, row or field enforcement
Observability	Commit markers, source positions, health evidence
Versioning	Semantic version, compatibility, attestation validity

21.2 Conformance

Declarations are claims until verified. Provider certification uses deterministic fixtures, fault injection, rate-limit behavior, schema evolution, token expiry, duplicate delivery, partial failure, and recovery. Production observations update confidence but do not redefine semantics. A capability whose behavior contradicts its declaration is quarantined from new planning and may invalidate active plans.

Decision protocol — Capability selection

Required inputs: Contract obligations; provider declarations and attestations; endpoint configuration; health; quotas; placement; statistics; current plan; tenant preferences.

Decision algorithm: Filter by required operation and semantic properties; intersect with endpoint and placement constraints; require valid conformance evidence; compute feasible capacity; pass surviving bindings to cost optimization.

Invariants: Brand preference cannot override semantics. Health is necessary but not sufficient. Capability versions are pinned in the plan.

Failure modes: No provider satisfies obligations yields No Valid Plan. Stale attestation yields Inconclusive or revalidation. A provider incident triggers runtime policy and replanning, not semantic substitution.

Tradeoffs: Strict certification slows provider introduction. A generic lowest-common-denominator capability simplifies support but forfeits useful native semantics; the model therefore allows precise optional capabilities.

21.3 MCP relationship

MCP tools and resources are exposure or acquisition capabilities, not a correctness boundary. Tool discovery is dynamic, so compiler plans pin the observed capability description and authorization context. A tool with side effects is modeled as an action with idempotency and approval obligations. Resources used as context carry freshness, entitlement, and provenance constraints like any other consumer surface.

21.4 Conformance tiers and promotion

Table 26. *Provider conformance tiers*

Tier	Evidence and permitted use	Planner restrictions
Certified	Full deterministic suite, fault injection, recovery, semantic proof notes, and provider-pair regression. Target: 4-8 provider-engineer months plus independent review for a materially new semantic family.	Eligible for §15.2 substitutions on either side, subject to the exact rule and evidence. Any contract risk tier.
Provisional	Reduced suite covering acquisition correctness, checkpoint/replay, deletion declaration, and failure classification. Target: 1-2 provider-engineer months for an already understood API family.	Snapshot read and idempotent merge only; maximum T1. Cannot appear on either side of a §15.2 substitution. All untested semantics remain Unknown.
Opaque-wrapped	Observed legacy behavior and measurement only; no provider semantic claim beyond the wrapper boundary.	Only contracts that explicitly tolerate the resulting Unknown. No substitution, autonomous optimization, or high-risk activation.

Promotion is evidence-driven. Opaque-wrapped becomes Provisional only after a version-pinned declaration, reduced-suite pass, failure taxonomy, and named provider owner. Provisional becomes Certified only after the full suite, proof-note review, negative and mutation fixtures, operational recovery evidence, and regression against every rule and provider pair for which eligibility is requested. A semantic version change, contradictory production observation, or expired attestation demotes or quarantines the affected capability; elapsed tenure never promotes it.

22 State Migration

22.1 State ownership model

Every stateful contract partition has one unfenced owner identified by contract, partition, and monotonically increasing epoch (INV-05, INV-06). Durable state includes source positions, checkpoints, deduplication records, pending commits, schema version, watermark, and replay horizon. Ownership is granted by a renewable lease backed by fencing; expired owners cannot commit.

Live migration: two owners over time

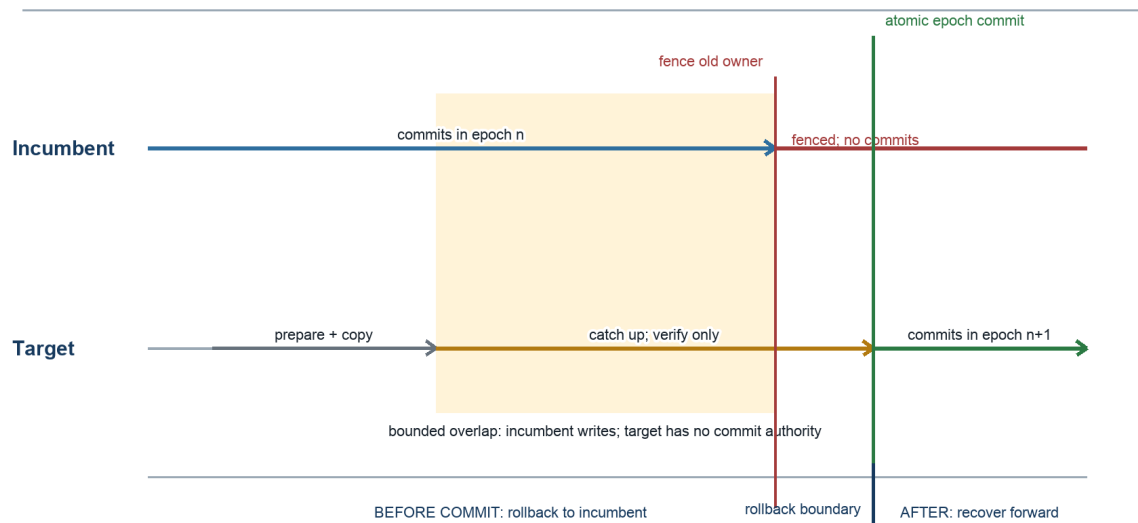


Figure 4. Live state migration with fencing, catch-up, and epoch commit.

22.2 Migration protocol

Migration prepares target state from a consistent source checkpoint, establishes a cut position, transfers or reconstructs state, replays to catch up, verifies equivalence, fences the old owner, atomically commits the new epoch, and then drains old resources. Before the atomic commit, the incumbent remains authoritative (INV-15); afterward, recovery moves forward and never resurrects the fenced owner (INV-16). A new replication slot may be dual-read only when duplicate handling is proven; otherwise the existing slot is handed over under a contract-bounded pause.

Table 27. State classes and migration method

State	Preferred method	Critical check
Source offset or LSN	Fenced handoff at committed position	No gap; duplicate interval bounded
Replication slot	Lease transfer or verified dual-read	Retention pressure and unique owner

State	Preferred method	Critical check
Checkpoint	Snapshot plus incremental log	Schema and operator compatibility
Deduplication index	Snapshot with overlap horizon	Covers maximum retry and replay window
Pending sink commit	Complete under old epoch or recover by transaction identity	No ambiguous double commit
Watermark	Monotonic transfer	Never advances past unprocessed data

Decision protocol — Migration admission

Required inputs: Old and new plans; semantic equivalence proof; state compatibility; source and sink fencing capabilities; estimated transfer; freshness budget; rollback window; current health.

Decision algorithm: Choose stop-and-copy, snapshot-and-catch-up, or dual-run only from the legality matrix; simulate worst-case overlap; verify capacity; prepare rollback; require canary evidence; schedule or execute by authorization mode.

Invariants: The migration method cannot weaken delivery semantics. The new epoch commits once. Rollback cannot reuse an epoch. Ambiguous sink commits are resolved before ownership transfer.

Failure modes: Loss of fencing, insufficient replay retention, or unverifiable state blocks migration. Failure before epoch commit leaves old owner active; failure after commit recovers new owner or performs a new forward epoch.

Tradeoffs: Stop-and-copy is simple but increases downtime. Dual-run reduces downtime but amplifies source load and duplicate risk. Snapshot-and-catch-up needs retained change history.

22.3 Backfill and reprocessing epochs

Backfill is not a rewind of the live epoch. A reprocessing epoch has separate input bounds, contract and transformation versions, state owner, checkpoints, deduplication namespace, evidence stream, and cost envelope. It reads a version-pinned historical range and writes an isolated generation. Consumers see no partial backfill: visibility changes through an atomic generation switch, a versioned partition replacement, or an explicitly declared merge protocol. The live epoch continues unless source or destination limits make concurrency infeasible.

Admission proves that historical inputs, replay retention, destination replacement semantics, and deduplication horizons cover the range. A reprocessing epoch cannot advance the live watermark, reuse a live idempotency namespace, or erase evidence for previously visible values. If the range is unavailable or the target cannot isolate and commit the replacement, compilation returns No Valid Plan with the missing capability in its unsatisfied core.

23 Scalability

23.1 Scale dimensions

The initial design point is the first ten enterprise customers, measured from their inventoried contracts, metadata churn, bytes, state, and provider quotas. The partitioning model below is a path that does not preclude larger deployment; it is not evidence for an unimplemented scale claim. Capacity gates are established from pilot measurements before each order-of-magnitude expansion.

Table 28. *Partitioning model*

Plane	Primary key	Cross-partition operation
Contract store	Tenant and contract identifier	Versioned reference only
Compilation	Compilation dependency hash	Content-addressed cache lookup
Knowledge Graph	Tenant, authority, semantic entity	Snapshot assembly
Coordination	Contract partition and epoch	Bounded migration transaction
Runtime	Contract partition and data key	Explicit exchange node
Evidence	Tenant, contract, time bucket	Asynchronous indexed projection

23.2 Avoiding global coordination

No global scheduler serializes executions. Placement is hierarchical: admission allocates a regional envelope, runtime cells assign partitions locally, and workers pull leased work. Contract activation and state-epoch ownership coordinate only one contract partition. Tenant-wide policy and key transitions are the coordination limit: they use a versioned barrier per tenant, not a global barrier, and are rate-limited through the invalidation queue in §27.5. A transition that cannot reach its high-risk deadline fences affected new access; lower-risk unaffected work continues under its prior envelope.

23.3 Fairness and hot keys

Per-tenant token buckets and weighted fair queues bound compute, network, and provider calls. Hot partitions may be split only when ordering and state semantics permit. Unsplittable hot keys receive isolated capacity or produce a feasibility violation; the system does not pretend horizontal scaling can solve an inherently serial contract.

24 Failure Handling

24.1 Failure domains

Failures are classified as transient infrastructure, provider throttling, permanent configuration, semantic incompatibility, policy invalidation, state corruption, ambiguous external commit, control-plane partition, relay loss, and regional loss. Classification determines whether retry is safe. Generic retry is not a correctness policy.

Table 29. Failure response matrix

Failure	Default response	Never do
Transient stateless worker	Retry from durable checkpoint	Advance watermark without commit
Provider throttle	Backpressure and bounded retry	Unbounded concurrency escalation
Contract-neutral schema change	Record neutrality proof; continue without pause	Bind a newly visible field implicitly
Bound field/type incompatibility	Pause affected partition and recompile	Coerce silently
Ambiguous sink commit	Resolve by transaction or dedupe identity	Blindly repeat non-idempotent action
Policy invalidation	Fence new access; apply contract safety mode	Continue on cached permission
State checksum failure	Quarantine and reconstruct from trusted point	Use suspected state
Control-plane outage	Continue valid leased work within bounds	Accept unsigned new work

Decision protocol — Retry classification

Required inputs: Failed operation; provider semantic declaration; idempotency key support; commit evidence; retry count and age; contract deadline; failure class.

Decision algorithm: Determine whether the operation is read-only, idempotent, transactionally identifiable, compensatable, or non-repeatable; inspect commit evidence; retry only when the resulting history remains allowed; otherwise enter Resolve Commit, compensate, or require operator decision.

Invariants: Retries cannot create an unbounded duplicate interval. Deadline and attempt budgets are enforced. Non-repeatable actions are never retried without positive non-commit evidence.

Failure modes: Incorrect provider declarations can make retry unsafe and trigger quarantine. Missing commit evidence yields Indeterminate, not success or failure.

Tradeoffs: Conservative indeterminate handling can require manual resolution. Strong idempotency infrastructure adds state but removes ambiguity.

24.2 Recovery testing

Every stateful graph is tested against worker loss, duplicate delivery, delayed messages, state-store failover, expired credentials, provider throttling, schema change, and migration interruption. Regional recovery is exercised with restored checkpoints and fenced epochs. A backup existing is not proof; recovery time and data-loss bounds are measured.

24.3 Contract-neutral schema evolution

An additive or descriptive schema delta is contract-neutral only when dependency analysis proves that it does not change a bound field or ancestor type, selected record set, key, delete marker, temporal field, transformation, policy classification, consumer schema, provider decoding semantics, or any allowed history. The analysis compares the old and new schema under the active IIR and emits a neutrality certificate bound to both schema versions. Such a change updates observed metadata and continues without pausing or recompiling the execution graph. A change that touches any dependency, or whose effect is Unknown, pauses only affected partitions and recompiles; no implicit field selection or coercion is permitted.

Part V — Enterprise Control

25 Security Architecture

25.1 Trust boundaries

Trust is split among tenant users, compiler control plane, managed runtime cells, customer relays, capability providers, external metadata authorities, and consumer surfaces. A signed plan authorizes a bounded data flow; it is not a bearer credential (INV-12). Data-plane access uses short-lived, least-privilege grants scoped to contract, capability, resource, operation, region, and epoch; secrets never enter compiler artifacts or model context (INV-13).

Security boundaries and scoped authority

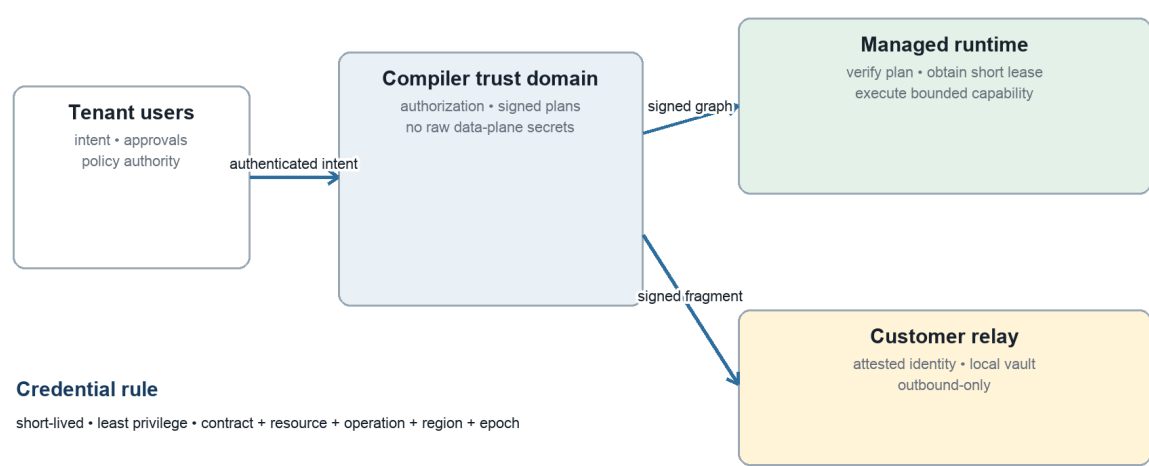


Figure 5. Trust boundaries and credential flow.

Table 30. Security controls

Area	Architectural control	Failure posture
Tenant isolation	Separate authorization context, encryption domain, quotas, and cache keys	Fail closed
Secrets	External vault references and short-lived leases; no secrets in graphs or logs	Revoke and re-authenticate
Plan integrity	Content address, compiler signature, runtime verification	Reject
Relay identity	Tenant-bound attested identity with rotation	Quarantine
Supply chain	Reproducible artifacts, signatures, staged updates, provenance	Block promotion
Data protection	Encryption, minimization, redaction, approved spill locations	Pause or quarantine
Administration	Separation of duties and just-in-time elevation	Deny and audit

Decision protocol — Secret grant issuance

Required inputs: Signed graph; runtime or relay identity; state epoch; provider resource; requested operation; tenant policy; current revocation state.

Decision algorithm: Verify graph and identity; prove requested capability is a subset of the plan; bind grant to resource, operation, region, and short expiry; record issuance; prefer provider-native delegation or local vault resolution.

Invariants: No raw secret enters contract, IIR, graph, telemetry, or model context. Grants are non-transferable and expire before plan authorization. Rotation does not require semantic recompilation unless capability changes.

Failure modes: Vault or identity outage prevents new grants. Suspected compromise revokes the epoch and fences work. Cached credentials beyond expiry are unusable.

Tradeoffs: Very short leases reduce exposure but increase dependency on identity availability. Local relay resolution limits secret movement but complicates centralized rotation evidence.

25.2 Isolation model

Control-plane storage uses tenant-scoped authorization at every access and tenant-derived encryption contexts. Runtime cells may be multi-tenant only when process, network, state, cache, telemetry, and cryptographic isolation are independently enforced. High-assurance tenants can require dedicated cells or relays as a placement constraint, with cost reflected explicitly.

Governance

26.1 Policy compilation

Policies enter as versioned facts from existing authorities. The compiler translates applicable policy into information-flow constraints, placement restrictions, transformations, retention obligations, approval requirements, and evidence obligations. It records which authority and rule produced each constraint.

Table 31. Governance enforcement points

Stage	Enforcement
Intent resolution	Hide unauthorized concepts and flag purpose conflicts
Static analysis	Track sensitivity, residency, retention, and consumer flow
Planning	Exclude providers and locations that cannot enforce obligations
Deployment	Issue least-privilege grants and approved state locations
Runtime	Apply masking, deletion, access, and evidence obligations
Consumption	Authorize resources, tools, queries, and data products
Change	Run impact analysis before contract or policy activation

26.2 Retention and deletion

Retention applies to destination data, runtime state, replay logs, dedupe records, samples, shadow outputs, telemetry payloads, and audit evidence. Each class has an explicit legal basis, period, and deletion capability. If a provider cannot delete with the contract's required semantics, it is infeasible as a target.

26.3 Deletion propagation protocol

Table 32. Deletion propagation states

State	Transition condition	Required evidence
Observed	Authorized source delete signal is durably identified	Source identity, object scope, observation time, position
Scoped	All contract-derived copies and state classes are enumerated	Lineage snapshot, exceptions, legal basis by class
Executing	Delete or legal-basis-specific retention action dispatched	Target operation identity and deadline
Awaiting proof	Targets accepted work but completion is not yet evidenced	Provider receipt, retry state, unresolved targets
Complete	Every required target has positive completion or authorized exception	Target proofs, state compaction markers, completion time
Blocked	A required target cannot perform or prove deletion	No Valid Plan or runtime violation with unsatisfied target

The completion objective names the subject or record identity, destination copies, runtime state, replay logs, deduplication indexes, samples, shadow outputs, telemetry payloads, and audit material. Audit evidence is normally not erased when a legal obligation requires proof; it is minimized or cryptographically tombstoned so it no longer contains the deleted payload while preserving the fact and authorization of deletion. Runtime state compaction cannot discard a delete marker before every downstream replay horizon that could recreate the record has passed. The contract sets a deadline and escalation; completion is never inferred from dispatch alone.

The evidence record contains the triggering authority, scope calculation, per-target operation and result, exceptions with legal authority, compaction watermark, and terminal status. A provider that lacks the needed delete, tombstone, replacement, or verifiable completion primitive is excluded during feasibility. Discovery after activation produces Blocked and the contract's declared containment action; silent partial deletion is forbidden.

26.4 Lineage

Compile-time lineage states intended dependencies and transformations. Runtime lineage confirms observed versions, partitions, execution epochs, and outputs. Existing catalogs receive lineage through

supported mechanisms; the compiler retains only the provenance required for proof and revalidation. Lineage gaps that affect a hard governance obligation are failures, not cosmetic warnings.

27 Operations

27.1 Contract-centered operations

Operators observe whether a contract is conforming, at risk, violated, blocked, migrating, or awaiting evidence. Machine and task metrics remain available for diagnosis but are not the top-level status. A green runtime with no fresh output is not healthy for a freshness contract.

Table 33. Operational signals

Signal	Definition	Use
Conformance	All measured hard obligations currently satisfied	Primary health
Freshness debt	Current visibility lag minus target	Prioritization and alerting
Delivery uncertainty	Unresolved commit or duplicate interval	Safety response
Plan risk	Expiring capability, policy, capacity, or statistics evidence	Proactive revalidation
State safety	Lease, checkpoint, checksum, replay horizon	Recovery readiness
Evidence completeness	Required audit events persisted and indexed	Compliance posture
Economic variance	Measured versus forecast cost interval	Reoptimization

27.2 Observability model

Metrics aggregate contract-level service, capacity, and cost. Traces follow compilation and execution causality without exposing sensitive values. Logs are structured, bounded, redacted, and correlated by contract, plan, epoch, partition, and transaction identity. Evidence is append-only and tamper-evident; searchable projections are rebuildable.

27.3 Incident response

An incident record captures affected contracts, violated invariants, blast radius, current state owner, last known good checkpoint, policy impact, and permitted mitigations. Automated mitigation may retry, backpressure, fail over, quarantine, or roll forward only when the plan already proves those actions legal. Emergency semantic relaxation always creates a contract version and authorization record.

Decision protocol — Alert emission

Required inputs: Contract objective; measurement and uncertainty; violation duration; maintenance state; dependencies; suppression policy; recipient and escalation rules.

Decision algorithm: Evaluate contract predicates over event time; require configured persistence to avoid flapping; correlate shared causes; emit one contract incident with affected partitions; escalate by duration and risk, not raw event count.

Invariants: Suppression never changes conformance history. Unknown measurements cannot be reported as healthy. Alert decisions are reproducible from retained aggregates.

Failure modes: Telemetry loss produces the bounded degraded behavior in §27.4. Over-correlation can hide independent failures; under-correlation creates storms.

Tradeoffs: Persistence windows reduce noise but delay detection. Contract-specific thresholds are more accurate than infrastructure defaults but require correct intent.

27.4 Bounded degraded operation

Fail-closed applies immediately to new authorization, activation, semantic change, policy override, lease extension, and work whose grant has expired. It does not retroactively invalidate ongoing execution that was Conforming at the last valid control-plane observation and still holds a signed graph, unexpired lease, locally verifiable policy and key material, and data-plane evidence (INV-20). Such execution enters Degraded Observation, not Conforming. The v1 post-M0 targets, to be validated in the first coexistence pilot, are maximum unobserved durations of 15 minutes for T3, one hour for T2, four hours for T1, and 24 hours for T0; they are design targets, not measured SLOs. Tenant policy may shorten but not lengthen T3 beyond one hour. The shortest remaining graph, credential, policy, or lease expiry is the actual bound.

Without the control plane, the data plane may establish Running, Backpressured, Indeterminate Commit, Locally Violated, Quarantined, and Degraded Observation from signed graph validity, lease clocks, provider receipts, local measurements, checkpoints, and commit identities. It cannot establish globally Conforming, extend authorization, approve a migration, or clear a governance violation (INV-14). When the bound expires, acquisition and new commits stop at a safe checkpoint; already ambiguous commits continue only through their declared resolution protocol.

The operator UI shows one dependency incident rather than a contract alert storm, the affected contract count by tier and remaining safe-run time, which evidence is still local, grants approaching expiry, contracts already stopped, and the exact action at each deadline. Complete-or-fence means a customer-visible service interruption for any contract not revalidated in time: acquisition and new commits stop at the safe checkpoint, the contract becomes Blocked, and named owners are notified. An override requires contract-owner and policy-authority signoff, a time bound, written justification, and a new provenance-bearing contract or policy decision; an operator cannot silently extend the window. Contract timelines retain the Unknown interval and override evidence; recovery revalidates before restoring Conforming.

27.5 Bounded invalidation and revalidation

Policy, key, provider, and metadata changes produce a dependency-indexed invalidation set. The impact analyzer first proves structural exclusion using tenant, authority, resource, field, purpose, region, consumer, and capability edges; contracts with no path to the changed fact are not queued. Affected work is ordered by immediate authorization violation, risk tier, distance to the changed predicate, envelope expiry, and consumer exposure. Duplicate invalidations coalesce by contract and target snapshot.

An immediate deny or revocation fences affected new access before other work (INV-17). Other contracts continue under their previous envelope for the lesser of policy-specified grace, remaining evidence validity, or the §27.4 targets. The v1 post-M0 targets, to be measured in the first pilot, are: start all T3 revalidations within five minutes and complete-or-fence them within 15 minutes; start 99% of T0-T2 within 15 minutes and complete-or-stop them within the applicable grace. Admission reserves a tenant lane sized for at least 10% of active contracts per minute only when memoized pass results remain valid; broad dependency invalidation can erase that assumption. The pilot must measure invalidation fan-out, memoization hit rate, queue age by tier, and throughput under deliberately broad policy and provider changes. A larger fan-out is an explicit tenant coordination-limit incident, not hidden backlog.

28 Explainability

28.1 Explanations are compiler products

Every diagnostic, plan choice, rejected alternative, migration, and runtime violation has a structured explanation derived from decision records. A language model may restate that record, but the record itself is deterministic and independently inspectable.

Table 34. *Decision record fields*

Field	Purpose
Question	The exact decision being made
Inputs	Versioned facts, statistics, policies, and authority
Candidates	Considered choices and search bounds
Rejections	Failed legality, feasibility, or policy predicates
Cost intervals	Expected outcomes and confidence
Selection	Tie-break and optimization policy
Stability	Dwell time, hysteresis, and migration cost
Evidence	Verification and shadow results

Field	Purpose
Authorization	Recommendation, approval, or autonomous grant

28.2 Counterfactuals

A useful explanation states what minimal change would alter the result: a provider gaining CDC, a freshness target moving from 60 to 180 seconds, a residency exception, or additional capacity. Counterfactuals are computed by relaxing one constraint class at a time and solving again. They are labeled proposals and never applied automatically.

28.3 Limits

Some decisions depend on uncertain measurements or opaque provider behavior. The explanation must expose that uncertainty and avoid causal claims unsupported by evidence. A plan can be the best observed candidate without being globally optimal; search bounds and unmeasured alternatives remain visible.

29 AI Architecture

29.1 Allowed roles

Table 35. *AI use boundary*

Allowed	Required deterministic control
Intent parsing and clarification	Typed AST, ambiguity checks, authorization-filtered resolution
Schema and concept matching	Candidate set, confidence calibration, owner approval where material
Transformation generation	Parse, type, static analysis, tests, policy, approval
Documentation	Facts linked to compiler records and provenance
Failure explanation	Structured incident and decision evidence
Recommendations and hints	Legality, feasibility, cost, stability, authorization

29.2 Prohibited roles

A model does not execute per-record decisions inside the deterministic runtime unless the contract explicitly defines probabilistic inference as a data operation with acceptance criteria. It does not own commits, state epochs, authorization, policy overrides, retry safety, or plan legality. Model output never serves as the sole evidence for conformance.

29.3 Model isolation and data handling

Model context is minimized to authorized metadata and examples. Sensitive values are excluded unless the contract and tenant policy explicitly permit them. Prompts, retrieved context, model identity, model version, tool calls, and outputs are provenance-bearing compilation inputs with retention controls. Provider training or secondary use is prohibited unless separately authorized.

Decision protocol — AI suggestion acceptance

Required inputs: Suggestion; source provenance; confidence; affected semantic scope; tests; policy; risk class; human approval policy.

Decision algorithm: Parse suggestion into a typed candidate; compare semantic delta; run deterministic analysis and generated boundary tests; reject unsafe or unresolved effects; require approval for material mappings; store accepted artifact independently of model prose.

Invariants: Model confidence is not proof. Unparseable output has no effect. Accepted artifacts remain reproducible without the model.

Failure modes: Hallucinated fields, biased matches, prompt injection, or model unavailability produce rejection or structured fallback. External content is treated as untrusted input.

Tradeoffs: Strict validation limits automation but protects correctness. Richer context improves suggestions while increasing privacy and injection exposure.

Part VI — Adoption and Evolution

30 Migration and Coexistence

30.1 Coexistence first

Existing SnapLogic, Airbyte, Fivetran, Informatica, and MuleSoft estates continue operating. The compiler initially observes, imports semantics where recoverable, and wraps existing executions as capabilities. It does not claim that a legacy pipeline definition contains a complete contract; implicit assumptions are surfaced as Unknown and require evidence or confirmation.

Table 36. Migration modes

Mode	Description	Risk
Observe	Ingest metadata, lineage, schedules, outcomes, and failures	Low; no execution change
Wrap	Treat existing job as opaque capability with measured semantics	Hidden behavior remains
Shadow	Compile equivalent intent and compare outputs without cutover	Duplicate load
Strangle	Move bounded contract partitions or consumers by epoch	State boundary complexity
Replace	Retire legacy execution after verified equivalence and rollback window	Highest change concentration

30.2 Import to IIR

Importers may lower source definitions to IIR only for semantics they can establish. A mapping records exact, inferred, and unknown elements. Runtime-specific constructs without a semantic counterpart remain opaque capabilities or migration blockers. The IIR should not become a lossless archive of every legacy product; doing so would fossilize their accidental complexity.

Decision protocol — Legacy cutover

Required inputs: Recovered contract; output equivalence criteria; source and sink state; consumer compatibility; shadow evidence; rollback horizon; business change window.

Decision algorithm: Freeze semantic change; establish comparable input window; shadow or dual-publish to isolated target; compare records and service behavior; resolve differences; transfer state epoch; canary consumers; retain rollback until new plan crosses evidence threshold.

Invariants: Legacy and new writers never concurrently own the same non-idempotent sink scope. Cutover criteria are declared before comparison. Rollback does not discard post-cutover committed data.

Failure modes: Opaque transformations or missing historical positions can prevent equivalence. In that case migration requires a new contract and acknowledged semantic break, not a false equivalence claim.

Tradeoffs: Long shadow windows increase confidence and cost. Partitioned strangling limits blast radius but extends dual operations.

30.3 Organizational migration

Teams migrate ownership from pipeline operators to contract owners and provider maintainers. Operational dashboards run in parallel until contract-level signals are trusted. Existing catalogs remain authoritative; only compiler-specific contract and plan artifacts are new systems of record.

31 Risks

31.1 Principal risks

Table 37. Risk register

Risk	Impact	Mitigation or containment
Authoring ceremony exceeds tolerance	Users route around the platform or do not adopt	Derived risk tiers, recorded defaults, interactive latency budget, and legacy importers
Intent language too broad	Undecidable or opaque semantics	Constrain MVP language and reject unsupported constructs
Provider declarations inaccurate	Unsafe plan or retry	Conformance, production attestation, quarantine
Metadata authorities conflict	Wrong binding or policy	Provenance, precedence, explicit ambiguity
Cost model gaming	Biased plan or customer distrust	Visible inputs, independent measurements, audit
Autonomous oscillation	Repeated migration and instability	Hysteresis, dwell time, confidence, risk tiers
State protocol defect	Loss or duplication	Formal model, fencing, fault injection, narrow initial modes
Compiler centrality	Control-plane blast radius	Content-addressed artifacts, regional isolation, runtime leases
AI prompt injection	Unauthorized mapping or leakage	Untrusted-input isolation and deterministic gates
Scope expansion	Becomes catalog, scheduler, or warehouse	Non-goals and capability boundary reviews

31.2 Compiler correctness risk

The compiler becomes a high-value correctness boundary. It requires differential testing across versions, property-based generation of contracts and capabilities, proof-rule regression suites, deterministic replay, staged rollout, and the ability to keep active graphs running when a new compiler is unavailable. Compiler upgrades never rewrite active epochs without an explicit revalidation decision.

31.3 Business-model risk

If revenue depends on inefficient compute, optimization credibility is structurally compromised. Commercial design must make customer savings compatible with vendor sustainability. Architecture cannot solve pricing, but it can make manipulation observable by exposing candidate plans, measured cost, and rejection rationale.

32 Open Research Questions

32.1 Semantic equivalence

How can the curated rule table in §15.2 be widened safely without weakening soundness? Which useful subsets admit mechanized containment or equivalence proofs across snapshot, CDC, query, and materialization plans, and how should deletion, late data, external actions, and source-specific isolation enter that model without making planning intractable?

32.2 Constraint and cost interaction

How can the solver expose small unsatisfiable cores and actionable counterfactuals when constraints span policies, provider semantics, time, and capacity? How should cost intervals, correlated latency, sparse failure evidence, and tenant risk preference be composed?

32.3 Verified state migration

Can a small state-transfer calculus cover replication slots, log offsets, deduplication horizons, pending commits, and watermarks across heterogeneous systems? Which parts can be model checked, and which require provider-specific conformance evidence?

32.4 Knowledge Graph consistency

What snapshot model provides reproducible compilation across independent, eventually consistent metadata authorities without requiring global transactions? How should authority, staleness, and confidence propagate through derived facts?

32.5 Safe generated transformations

How can example-derived mappings be tested for semantic completeness rather than syntactic validity? Which metamorphic and boundary tests detect common enterprise mapping errors, especially around null, missing, deletion, currency, time zone, and identifiers?

32.6 Economic mechanism

Which pricing units preserve incentives for cost reduction while funding compiler, governance, and reliability work? Can an external audit protocol verify that plan ranking follows the declared tenant objective without disclosing proprietary provider pricing agreements?

33 Minimum Viable Platform

33.1 MVP scope

The MVP proves the compiler architecture on a deliberately narrow semantic subset: relational snapshot and CDC ingestion into an analytic destination; deterministic filter, projection, rename, cast, mask, and key-based upsert; bounded freshness; at-least-once acquisition with idempotent destination commit; one managed runtime region plus outbound relay; external catalog projection; contract-level conformance and provenance.

Table 38. MVP inclusions and exclusions

Included	Excluded until later
Typed contract and semantic IIR	General workflow and arbitrary control flow
Snapshot and log-based CDC	Non-repeatable external actions
Deterministic relational transforms	Probabilistic per-record transforms
One state migration protocol	Universal cross-provider live migration
Recommendation and approval optimization	Autonomous cost migration by default
External catalog and policy inputs	New catalog authoring experience
Dataset and governed MCP resource exposure	Broad operational tool execution
Single-source contracts	Multi-source joins and cross-source snapshot boundaries

Multi-source contracts are excluded because no honest MVP semantics exist for a join across independent clocks. The eventual model requires a per-source snapshot identity, a declared join-consistency class such as common-cut, bounded-skew, or latest-observed, and a maximum cross-source skew as a hard constraint. No planner support is added before corresponding source-position, late-data, and replay rules exist.

33.2 Milestone 0: executable thin thread

Before deepening the compiler, a four-to-six-person team delivers in six weeks one contract that snapshots one PostgreSQL table into one analytic destination. It uses one hard-coded legality rule, one deterministic projection, a single runtime process, and traced source-read-to-destination-visibility

evidence. There is no optimizer, relay, live migration, CDC, general solver, or autonomous behavior. The gate is a fresh transaction visible at the destination with contract, graph, and evidence linkage; this milestone validates the artifact boundaries, not the MVP.

33.3 Costed implementation sequence

Table 39. *Approximate implementation plan*

Step	Scope	Team / elapsed duration
M0	PostgreSQL snapshot thin thread and traced live transaction	4-6 engineers / 6 weeks
1	Contract semantics, AST, IIR, invariants, diagnostics	6-8 engineers / 12-16 weeks
2	Second provider with materially different capabilities	4-6 engineers / 8-10 weeks
3	Curated legality table, feasibility, unsat cores; no cost optimizer yet	6-8 engineers / 12-16 weeks
4	Deterministic runtime, conformance, single-epoch state ownership	8-10 engineers / 16-20 weeks
5	Outbound Relay, signed fragments, short-lived grants	5-7 engineers / 12-16 weeks
6	Stop-and-copy then catch-up migration with fault injection	6-8 engineers / 14-18 weeks
7	Cost intervals, shadow execution, recommendation and approval	6-8 engineers / 12-16 weeks
8	One coexistence pilot, importer, cutover evidence	5-7 engineers / 12-16 weeks

The steps overlap after interfaces stabilize; the estimate is approximately 18-24 months with a core team growing from 6 to 24-30 engineers, plus provider, security, SRE, product-design, and compliance support. It excludes provider certification beyond the first two providers, go-to-market staffing and enablement, and SOC 2 or ISO certification work. The estimate has low confidence before Milestone 0 and first conformance results. Funding is staged at M0, legality/feasibility, single-epoch runtime, and migration gates.

33.4 MVP exit criteria

The MVP exits only when a fresh contract can be authored conversationally, compiled with a reproducible proof record, deployed without customer cluster operation, exercised through a new live transaction, traced to terminal destination visibility, migrated between two legal plans without loss, and audited from contract through runtime evidence. Green infrastructure or pre-existing data is insufficient.

33.5 Who this is for and why the cost is justified

The program is fundable before broad connector coverage only where correctness, governance, and auditability are the purchase criteria. The initial buyers are: regulated enterprises operating under residency or evidentiary mandates; organizations responding to a material semantic-drift or governance

incident; and enterprises with a small number of data contracts whose failure has very high financial or operational consequence. An organization buying primarily on connector count, rapid departmental automation, or lowest ingestion price is not an initial buyer.

Table 40. *Initial buyer, incumbent approach, and unresolved failure*

Buyer condition	What runs today	Failure a connector leader does not remove
Residency or evidentiary mandate	Managed replication plus catalog, policy engine, tickets, and manual audit collection	No single artifact proves that every physical plan, placement, and runtime epoch remained within the declared obligation.
Recent semantic-drift or governance incident	Visual pipelines and connector jobs with engine-specific transformations and post hoc lineage	A legal-looking mode or engine change can alter histories; lineage does not prove substitution legality.
Few high-value contracts	Bespoke application jobs, warehouse tasks, CDC service, and operator runbooks	State ownership, ambiguous commits, and exception approvals are distributed across systems and cannot be replayed as one decision record.

The commercial consequence is a long, reference-led enterprise sale with architecture, security, and policy stakeholders. Adoption starts by observing and wrapping incumbent SnapLogic, Airbyte, Fivetran, Informatica, MuleSoft, or bespoke estates, then shadows and strangles bounded contracts under \$30. A mandatory rip-and-replace motion would defer evidence until too late and is inconsistent with the coexistence architecture.

34 Future Evolution

34.1 Semantic expansion

Later releases may add bidirectional synchronization, event routing, governed query, operational actions, and richer contracts only after defining observable semantics, capability evidence, static analysis, and migration rules.

34.2 IIR ecosystem

External systems may emit constrained IIR modules only if the representation proves stable and modules carry authority, effects, semantics, and proof obligations. A loose interchange format would recreate connector ambiguity.

34.3 Compiler federation

Domain-local compilers may exchange signed contract modules, proof summaries, and provider facts under central policy. Cross-domain composition requires explicit trust and policy authorities.

34.4 Mature autonomy

Autonomy expands by proof class: equivalent stateless placement first, stateful migration only after measured reliability, and non-equivalent change never without a new contract decision.

34.5 End state

The end state is a control plane where durable contracts define outcomes, compilers verify mechanisms, and small runtimes execute signed graphs without hiding failures or tradeoffs.

34.6 Decision requested of the reader

The requested decision is whether to fund Milestone 0 and the evidence-gathering work needed to decide on the full architecture, not whether to approve every future mechanism. Evidence that changes the design includes inability to meet the T0 five-minute authoring target, provider behavior that defeats sound rule-table admission, or operational measurements showing that bounded revalidation and leased execution cannot contain correlated dependency failure.

Three questions must be resolved before implementation beyond Milestone 0: (1) can a reviewed rule table and conformance suite maintain sound legality for the initial provider pair; (2) can contract and state identities survive a real traced snapshot without leaking execution mechanics into intent; and (3) can the product deliver T0 authoring within the stated question and latency budgets while preserving deterministic activation? Negative evidence narrows or revises the architecture before expanding provider count or autonomy.

Appendix A — Invariant Registry

These invariants are normative. Subsystems may strengthen them but must not define weaker local variants.

Table 41. Core invariants

ID	Invariant	Primary enforcement
INV-01	An active epoch names exactly one immutable contract version and semantic IIR.	Activation; graph validation
INV-02	Every observable history of an admitted plan is contained in the contract's allowed histories.	Semantic specification; verification
INV-02a	V1 admits only through a sound curated rule with discharged obligations; absent rules, Unknown obligations, or invalid evidence reject.	Rule table; proof notes; conformance
INV-03	Optimization, adaptation, and failure handling never relax hard constraints.	Solver; runtime policy
INV-04	No Valid Plan is preferable to an unproven or semantically degraded plan.	Compiler phase ordering
INV-05	A stateful contract partition has at most one unfenced owner per epoch.	Leases; fencing
INV-06	Epoch numbers increase monotonically and are never reused.	Coordination state
INV-07	A watermark never passes durably committed input effects.	Commit protocol
INV-08	Retries preserve allowed duplicate, ordering, and side-effect semantics.	Provider claims; retry classifier
INV-09	Every automatic decision replays from versioned inputs and a declared algorithm.	Decision record
INV-10	AI output cannot authorize, commit, migrate, or execute data effects.	Compiler/runtime trust boundary
INV-11	Catalog facts retain authority, observation time, validity, and provenance.	Knowledge Graph ingestion
INV-12	A signed graph grants no access beyond its verified plan and contract.	Grant issue; runtime validation
INV-13	Secrets never appear in contracts, IIR, graphs, telemetry, or model context.	Secret scan; grant architecture
INV-14	Unknown required evidence forbids Conforming; execution is limited to §27.4's last-valid envelope.	Conformance; degraded timer
INV-15	Before epoch commit, migration failure leaves the incumbent authoritative.	Migration coordinator
INV-16	After epoch commit, recovery moves forward under a new epoch and never revives a fenced owner.	Migration recovery
INV-17	Policy invalidation fences unauthorized new access before optimization or availability.	Policy revalidation
INV-18	Shadow execution emits no external action or production write.	Shadow placement; credentials
INV-19	Commercial preference cannot alter legality or falsify cost evidence.	Optimizer audit
INV-20	Control-plane failure does not invalidate signed, unexpired authority; §27.4 bounds ongoing work and no new authority is gained.	Control/data-plane separation

Appendix B — State and Migration State Machines

Table 42. *Partition ownership states*

State	Allowed transitions	Commit authority
Unassigned	Preparing	None
Preparing	Active or Aborted	None; reads for preparation only
Active	Draining, Fenced, or Failed	Current epoch owner
Draining	Fenced or Active rollback before epoch commit	Incumbent within bounded lease
Fenced	Retired	None
Failed	Recovering	None until fenced recovery lease
Recovering	Active in a new epoch or Quarantined	New epoch after verification
Quarantined	Preparing after operator repair	None

B.1 Replication slot ownership

A replication slot or equivalent retained-log lease is modeled as a scarce source resource. The plan names its owner, retention risk, restart position, and transfer method. During snapshot-and-catch-up, the incumbent may continue consuming while the target builds state from a duplicated read only if the source and contract permit it. Otherwise the transfer uses a fenced pause. A slot is released only after the new epoch has committed and rollback no longer depends on the retained position.

B.2 Ambiguous commit state

If the runtime cannot determine whether an external write committed, the partition enters Resolve Commit. Resolution uses a transaction identity, idempotency key, destination query, or provider-specific commit token. Until resolution, the watermark cannot advance and migration cannot commit. If the provider offers no resolution mechanism and replay is not contract-safe, an operator must adjudicate; the system cannot convert uncertainty into success.

Appendix C — Failure and Diagnostic Taxonomy

Table 43. *Compiler diagnostics*

Code family	Meaning	Typical disposition
PARSE	Source cannot form an AST	Clarify or correct
RESOLVE	Reference absent, ambiguous, stale, or unauthorized	Bind, refresh, or deny
TYPE	Structural, temporal, or effect mismatch	Correct contract
POLICY	Governance obligation violated or unknown	Reject or obtain authorized policy change
LEGAL	Physical substitution changes allowed histories	Remove candidate
FEAS	Capacity, reachability, quota, or evidence cannot satisfy obligation	No Valid Plan or measure
COST	Estimate missing or too uncertain for decision	Measure or retain incumbent
VERIFY	Candidate fails tests, conformance, or dry-run	Reject candidate
MIGRATE	State cannot transfer safely	Retain incumbent or require planned downtime

Table 44. *Runtime conformance states*

State	Definition
Conforming	All hard obligations measured and satisfied
At risk	Evidence or capacity is approaching expiry or limit
Degraded Observation	Last-valid execution continues inside §27.4 while global conformance is Unknown
Unknown	A required measurement or authority is unavailable and no bounded last-valid claim is applicable
Violated	A hard obligation is outside its permitted envelope
Blocked	Execution cannot proceed without unsafe behavior
Indeterminate	External effect may or may not have committed
Quarantined	Execution is fenced due to integrity or security concern

Appendix D — Glossary

Table 45. Glossary

Term	Definition
Allowed history	A sequence of externally observable data values, visibility times, ordering, effects, and violation signals permitted by a contract.
Capability	A typed operation with declared semantics, limits, placement, security behavior, and evidence.
Capability Provider	An implementation that exposes one or more capabilities for an external system and passes conformance checks.
Compilation snapshot	The versioned set of metadata, policy, capability, statistics, and authority observations read by one compilation.
Conformance envelope	The contract, plan, evidence, and validity bounds against which a running epoch is evaluated.
Contract partition	The smallest independently owned and recoverable unit of execution state under a contract.
Epoch	A monotonically increasing ownership generation for a contract partition.
Execution Graph	A signed, immutable, disposable graph of deterministic runtime operations for a verified plan and epoch.
Feasibility	Whether a legal plan can meet all hard obligations given capacity, reachability, quotas, and evidence.
Fencing	A mechanism that prevents an obsolete state owner from committing after ownership changes.
Integration Contract	The durable, versioned declaration of allowed outcomes, service objectives, governance, and failure behavior.
Contract risk tier	A compiler-derived T0-T3 classification controlling ceremony, verification, grace, and automation; it never relaxes hard semantics.
Intent AST	A source-provenance-preserving, typed representation of user intent before lowering.
IIR	Intent Intermediate Representation: a versioned compiler representation separating semantic, logical, distributed, and deployment concerns.
Knowledge Graph	A provenance-bearing compiler projection of facts from existing enterprise authorities and observations.
Legality	Whether every observable history of a physical plan is contained in the contract's allowed histories.
No Valid Plan	A compiler result proving that no considered plan satisfies all hard constraints, with an unsatisfied core and counterfactual proposals.
Physical Plan	A compiler-selected binding of logical obligations to capabilities, placement, partitioning, state, and failure protocols.
Relay	A minimal outbound-only runtime component used to reach private enterprise systems.
Shadow execution	Isolated execution used to measure and compare a candidate without production effects.
State owner	The single unfenced holder authorized to advance durable state for a partition and epoch.
Watermark	A claim that input up to a position or event time has been processed according to the commit protocol.

Appendix E — Worked T0 Activation Summary

Illustrative compiler artifact, not implementation evidence. Contract C-1047 snapshots PostgreSQL public.orders to analytics.raw.orders. References and credentials resolved uniquely; activation asked zero questions. Provider evidence covers consistent snapshots, key-based idempotent merge, and keyed deletion.

Table 46. Activation summary for contract C-1047

Field	Activated value	Rationale	Authority	Review queue
Scope	Current rows in public.orders; key order_id	Unique authorized dataset and stable primary key	PostgreSQL schema observation and data owner	On schema, key, or ownership change
Risk tier	T0	No regulated/secret data, restricted border, external effect, production target, or multiple consumers	Risk decision table v1 plus policy snapshot	On classification, effect, environment, or consumer change
Freshness	24 h: source snapshot observation to destination visibility	Tenant-approved T0 default	Tenant T0 defaults v1	Review within 30 days or on SLA request
Delivery	At least once; idempotent merge on order_id	Retry preserves final-row semantics	Certified provider attestations	On provider semantic version or contradictory observation
Deletion	Propagates at next successful snapshot by deleting destination keys absent from the complete source snapshot	Target supports keyed delete and compiler retains the prior key set	Contract default policy plus provider proof	On loss of complete-snapshot or delete capability
Retention	30 days in destination	Matches source retention policy; no broader copy retention	Policy authority snapshot P-88	Before policy expiry or requested extension
Environment	Non-production	Tenant-approved T0 activation default	Tenant T0 defaults v1	Approval required before production promotion
Consumers	Requester only	No additional exposure was requested	Requester identity and entitlement snapshot	On share, role, or purpose change
Activation	Epoch 1; snapshot plan; zero open questions	All defaults, capability evidence, and policy inputs valid	Compiler decision record DR-1047	30-day owner review; immediate review on invalidation

Appendix F — Technical References

The following primary references provide historical and technical grounding for declarative execution, intermediate representations, distributed state, consistency, catalogs, capability protocols, and named hybrid-runtime comparisons.

- Dean and Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. OSDI 2004. <https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/>
- Chang et al. Bigtable: A Distributed Storage System for Structured Data. OSDI 2006. <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>
- Corbett et al. Spanner: Google's Globally-Distributed Database. OSDI 2012. <https://research.google/pubs/spanner-googles-globally-distributed-database-2/>
- Verma et al. Large-scale cluster management at Google with Borg. EuroSys 2015. <https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/>
- DeCandia et al. Dynamo: Amazon's Highly Available Key-value Store. SOSP 2007. <https://www.amazon.science/publications/dynamo-amazons-highly-available-key-value-store>
- Kreps, Narkhede, and Rao. Kafka: a Distributed Messaging System for Log Processing. NetDB 2011. <https://cwiki.apache.org/confluence/display/KAFKA/Kafka+papers+and+presentations>
- Lattner and Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. CGO 2004. <https://llvm.org/pubs/2004-01-30-CGO-LLVM.html>
- Apache Iceberg Table Specification. <https://iceberg.apache.org/spec/>
- Apache Polaris documentation. <https://polaris.apache.org/>
- Model Context Protocol architecture overview. <https://modelcontextprotocol.io/docs/learn/architecture>
- CockroachDB Architecture Overview and Transaction Layer. <https://www.cockroachlabs.com/docs/stable/architecture/overview/>
- OpenMetadata System Architecture. <https://docs.open-metadata.org/v1.12.x/developers/architecture>
- SnapLogic. Snaplexes and Groundplex architecture. <https://docs.snaplogic.com/snaplexes/snaplexes-about.html>
- Fivetran. Hybrid Deployment architecture. <https://fivetran.com/docs/core-concepts/deployment-models/hybrid-deployment>
- Airbyte. Connector Development Kit documentation. <https://docs.airbyte.com/platform/connector-development-kit/>